Progetto di macchine aritmetiche

(programma da 9CFU)

ARCHITETTURA *dei* **SISTEMI DIGITALI**

Prof.ssa Alessandra De Benedictis
(prof. Mazzocca)

Progettazione di una macchina aritmetica

Realizzazione dell'operazione:

- Approccio combinatorio
- Approccio sequenziale

Realizzazione del circuito che esegue l'operazione complesse:

- Approccio a livello porta
  - Circuito ad hoc per ciascuna operazione.
- Approccio a livello comportamentale
  - Perdita di controllo sull'implementazione fisica.
- Approccio strutturale
  - Potrebbe integrare componenti a livello porta e componenti a livello comportamentale.

In taluni casi si ricorre a memorie ROM…

Riutilizzando componentistica disponibile (memorie), è possibile cablare all'interno delle righe di una memoria i risultati di un'operazione specifica (solo ad un operando).

Esempio: A^2

ADDRESS	VALUE
0	0
1	1
2	4
3	9
4	16
…	…

Presentazione dei dati

- Presentazione **parallela**
  - I bit degli operandi sono presentati in ingresso e i risultati sono calcolati *contemporaneamente* (a meno dei ritardi di programmazione).
- Presentazione **seriale pura**
  - Alle linee di ingresso i bit degli operandi sono presentati sequenzialmente nel tempo e analogamente i bit del risultato appaiono in uscita *sequenzialmente*.
  - Little-endian; big-endian ha senso per la divisione ma non per somma e moltiplicazione.
- Presentazione **seriale a gruppi di bit**
  - Una voce è divisa in gruppi di bit: i gruppi omologhi sono presentati in parallelo mentre gruppi della stessa voce sono applicati in serie.
- Presentazione **mista**
  - Uno dei due operandi è presentato in parallelo, mentre l'altro con modalità seriale.

Tipo di presentazione vs unità aritmetica

Il tipo di presentazione influisce sull'unità aritmetica:

- **Natura** dell'unità aritmetica:
  - Rete combinatoria per presentazione parallela
  - Rete sequenziale negli altri casi
- **Complessità**
  - Seriale è più piccola ed economica di una parallela
- **Velocità**
  - Seriale richiede molti impulsi di clock
  - Parallela è più veloce (solo ritardi di una rete combinatoria)

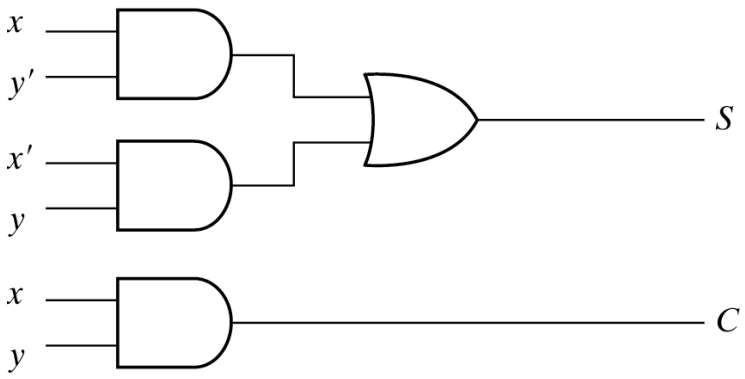
OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORI

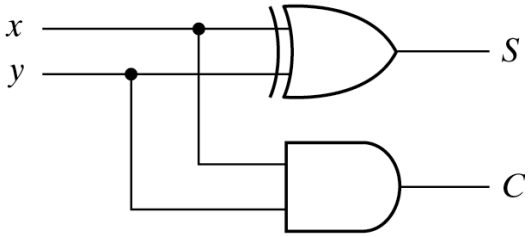
Half adder

Componente di base per la somma di due bit:

x	y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1



(a) $S = xy' + x'y$
$C = xy$



(b) $S = x \oplus y$
$C = xy$

Fig. 4-5 Implementation of Half-Adder

Full adder

Componente di base per la somma di tre bit:

x	y	c_{in}	S	C_{out}
0	0	0	0	0
0	0	I	I	0
0	I	0	I	0
0	I	I	0	I
I	0	0	I	0
I	0	I	0	I
I	I	0	0	I
I	I	I	I	I

$$\mathbf{S} = x'y'c_{in} + x'yc_{in}' + xy'c_{in}' + xyc_{in} = x \oplus y \oplus c_{in}$$

$$\mathbf{C_{out}} = xy + xc_{in} + yc_{in} = xy + c_{in}(x+y)$$

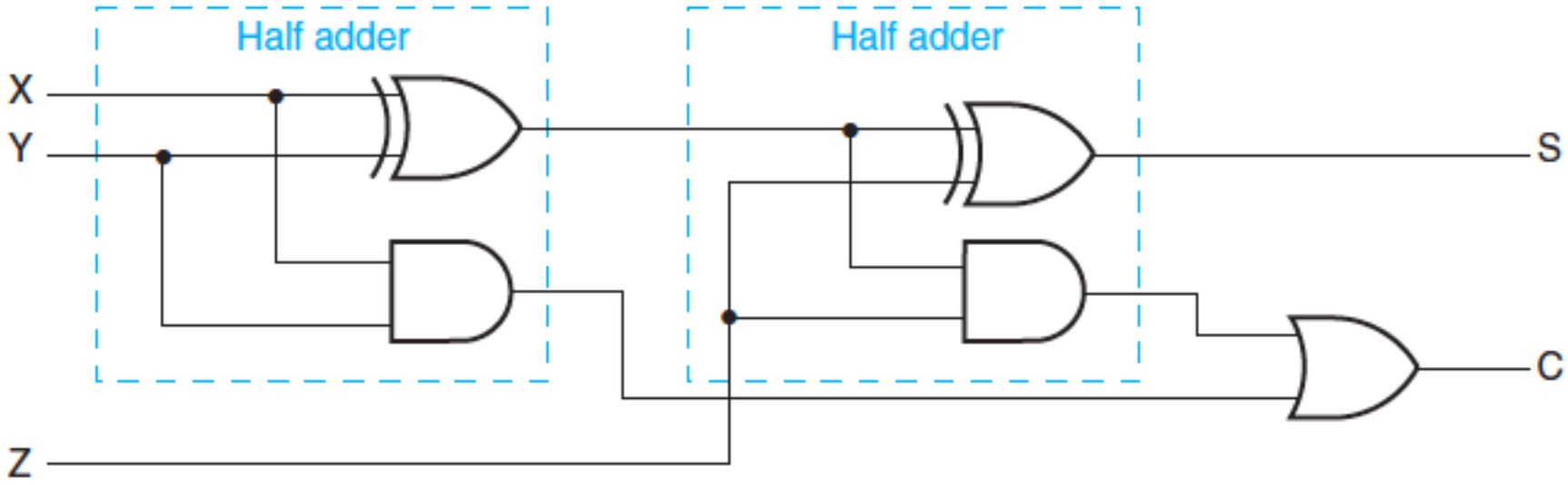
Si può scrivere come → $\mathbf{C_{out}} = xy + c_{in}(x \oplus y)$

Realizzazione di un FA a partire da due HA

Full adder realizzato mediante due half adder ed una porta OR per la composizione del riporto uscente secondo le espressioni precedentemente mostrate:

$C_{HA} = xy$ $\qquad$ $C_F = xy + z(x \oplus y)$

$S_{HA} = x \oplus y$ $\qquad$ $S_F = x \oplus y \oplus z$



Implementazione VHDL data-flow di un FA

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity full_adder is
        port
        (       a,b, c_in: in std_logic;
                s, c_out: out std_logic
                );
end full_adderadder;
architecture dataflow of full_adder is


 begin
        s <=((a xor b) xor c_in);
        c_out <= ((a and b) or (c_in and (a xor b)));
 end dataflow;
```

$$\mathbf{S} = a \oplus b \oplus c_{in}$$

$$\mathbf{C_{out}} = ab + c_{in}(a \oplus b)$$

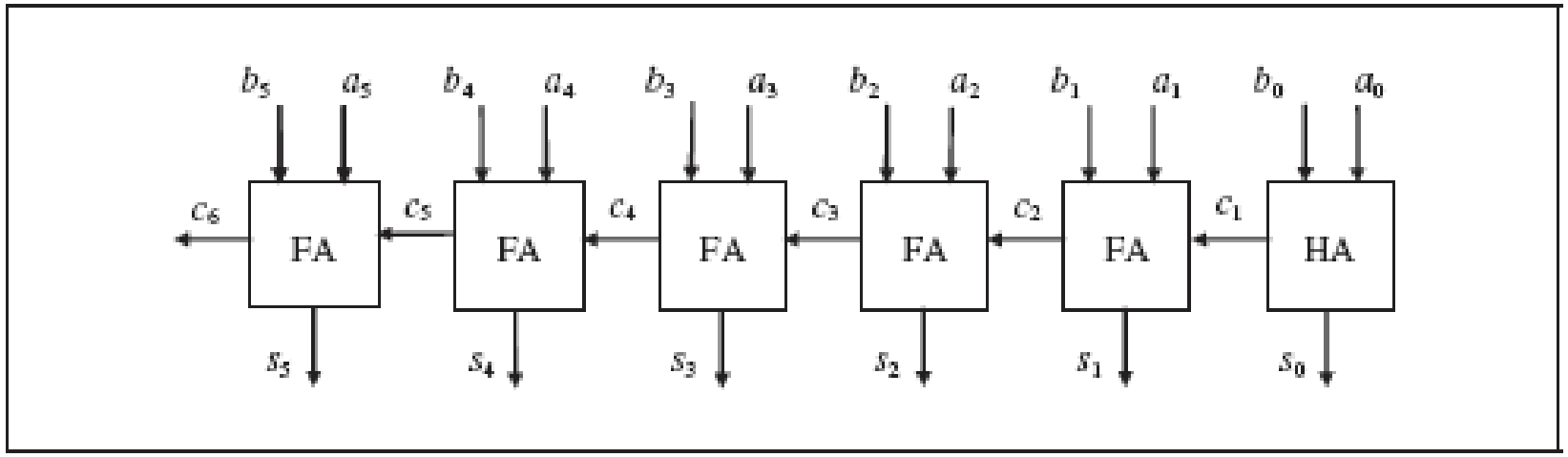
OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORI A PROPAGAZIONE DI RIPORTO

Ripple Carry Adder

La struttura dell'RCA deriva dall'algoritmo manuale per la somma di due numeri interi positivi di n bit ciascuno:

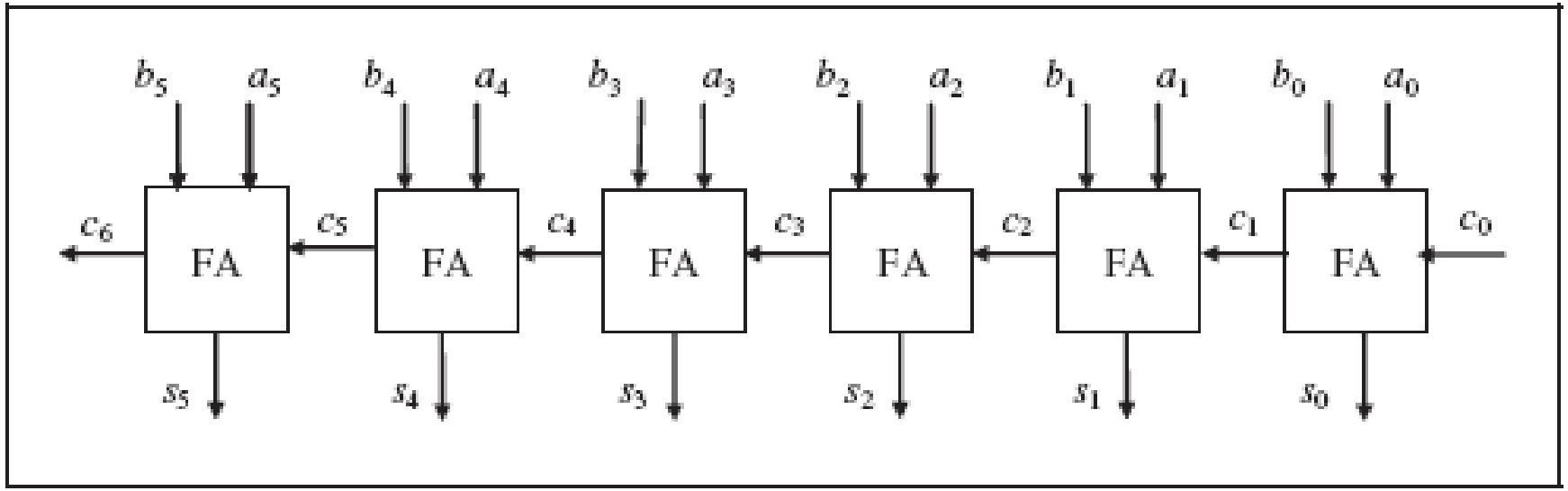
- la cella a destra riceve i bit meno significativi dei due addendi e produce la somma s0 ed il riporto c1 in entrata alla seconda cella;



- la seconda cella da destra riceve i bit a1 e b1 dei due addendi ed il riporto c1 dallo stadio precedente e produce la somma s1 ed il riporto c2 allo stadio successivo, e così via.

Ripple Carry Adder costituito da n full adder

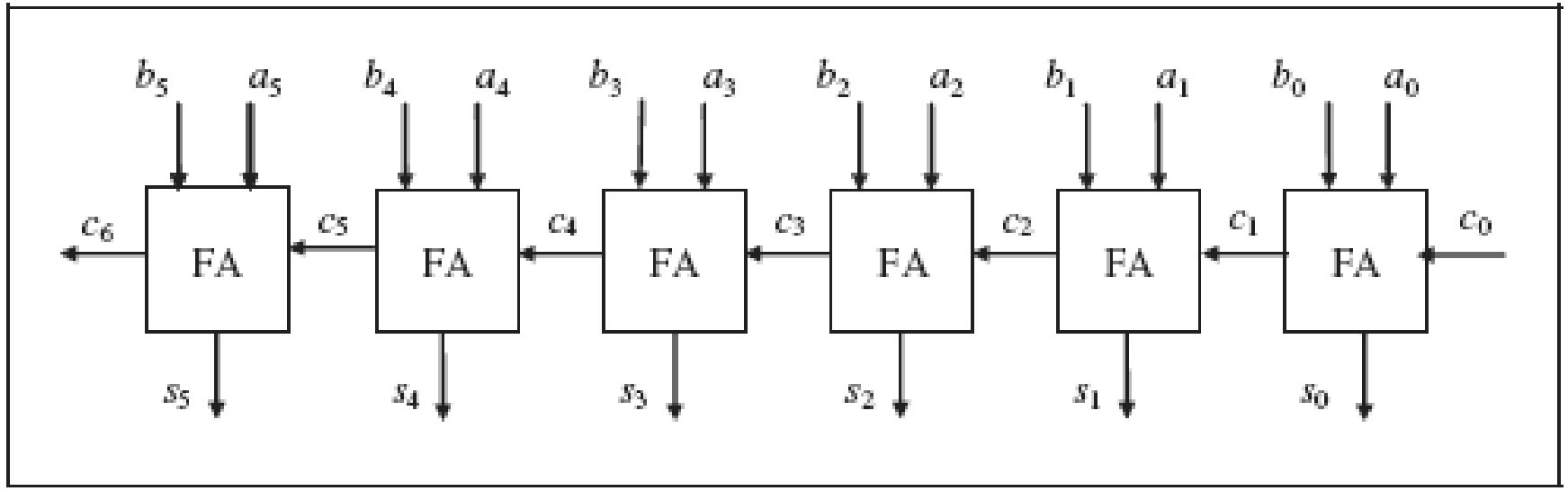
Ponendo c0=0 è possibile ottenere una struttura più regolare costituita da n dispositivi full adder identici connessi in serie.



Il circuito è in grado di eseguire somme algebriche tra numeri rappresentati in complemento a 2 e determinare l'eventuale condizione di overflow:

$$b_n' * a_n' * c_n + b_n * a_n * c_n'$$

Ripple Carry Adder: ritardo ed area



- Il ritardo è pari a 2*Δ***n** per la somma di operandi da n bit, se Δ è il ritardo di porta;
- L'area occupata è di 5*n porte logiche, dove 5 è il numero di porte logiche necessarie per ogni FA.

OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOTTRATTORE

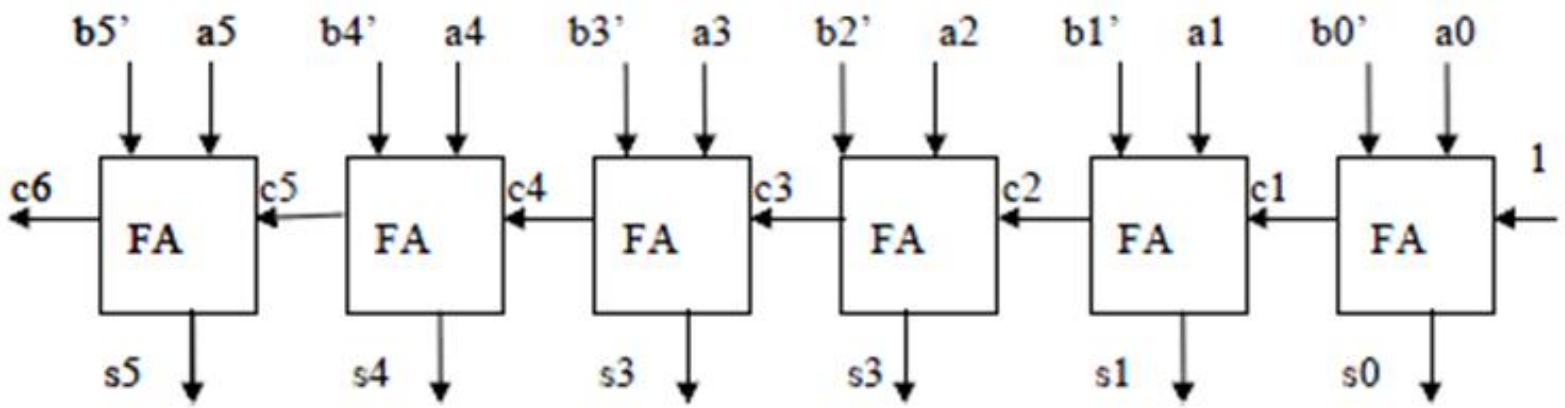
Sottrattore su n bit basato su Ripple Carry Adder

Effettuare l'operazione di sottrazione **A-B** con A e B entrambi addendi su n bit equivale ad effettuare l'operazione di addizione fra A e il complemento a 2 di B:

$$A+(-B)$$

Il complemento a 2 di B viene ottenuto aggiungendo 1 al complemento diminuito di B:

$$-B = b'_{n-1}b'_{n-2}\ldots b'_0 + 1$$



OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORE/SOTTRATTORE

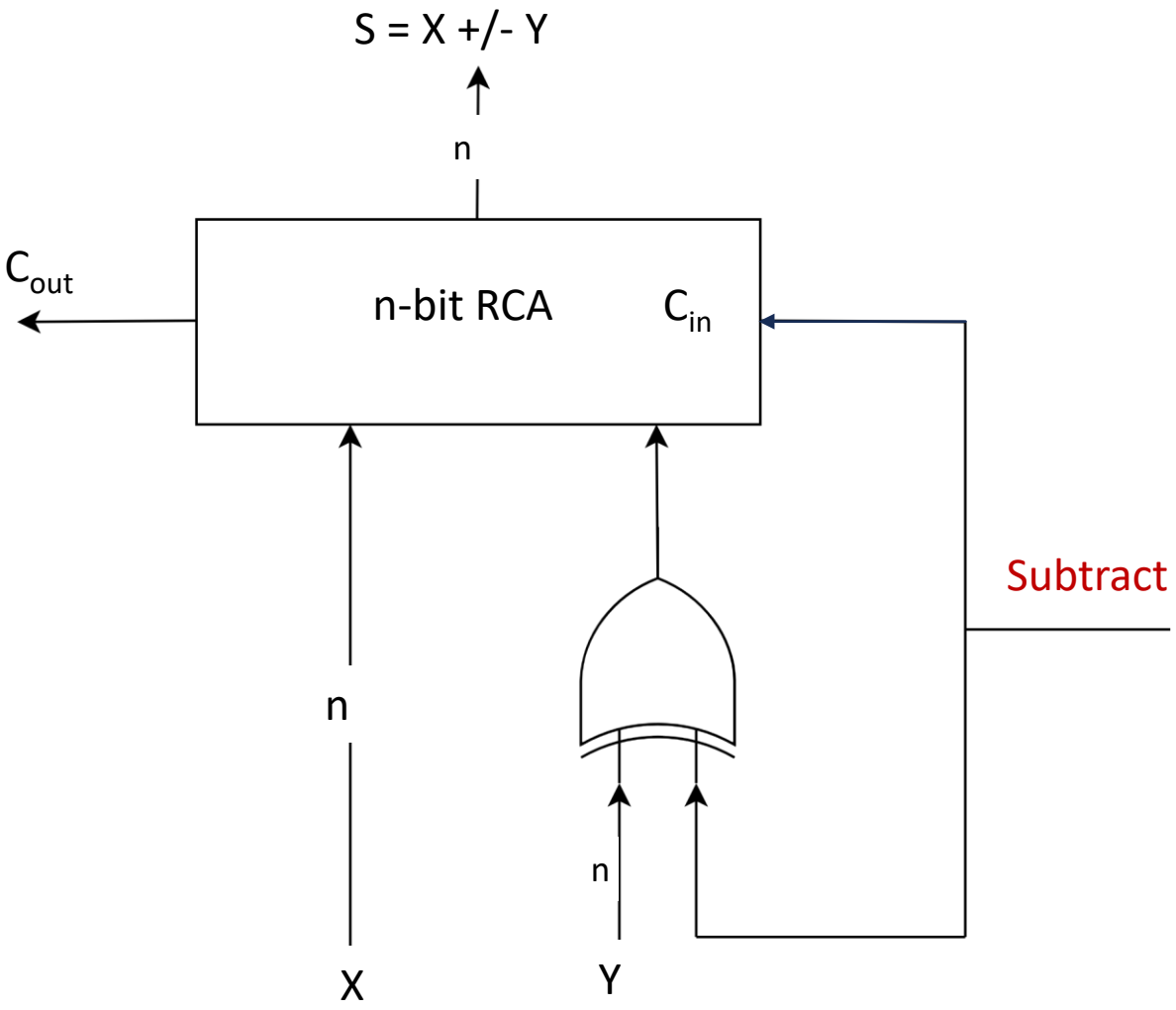
Circuito addizionatore/sottrattore su n bit

Proprietà della porta XOR:

- Y = Y xor 0
- not Y = Y xor 1

Operazione di sottrazione:

- X - Y= X + (-Y)= X + (notY+1)= X + notY + 1



Implementazione strutturale dell'adder/subtractor in VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity adder_sub is
        port( X, Y: in std_logic_vector(7 downto 0);
                cin: in std_logic;
                Z: out std_logic_vector(7 downto 0);
                cout: out std_logic);
end adder_sub;
architecture structural of adder_sub is
        component ripple_carry is
        port( X, Y: in std_logic_vector(7 downto 0);
                c_in: in std_logic;
                c_out: out std_logic;
                Z: out std_logic_vector(7 downto 0));
        end component;
        signal complementoy: std_logic_vector(7 downto 0);

        begin
        complemento_y: FOR i IN 0 TO 7 GENERATE
                                complementoy(i)<=Y(i) xor cin;
                                END GENERATE;
        RA: ripple_carry port map(X, complementoy, cin, cout, Z);
        end structural;
```

Il segnale C_{in} sarà mappato con il segnale subtract

Componente RCA (realizzato come composizione di FA)

Blocco che effettua la XOR bit a bit tra le cifre del secondo operando e il riporto entrante

OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORI CON VALUTAZIONE PARALLELA DEI RIPORTI

Limiti del sommatore RCA

- Il sommatore ripple carry presenta prestazioni scarse a causa della propagazione dei riporti (**il ritardo è pari a 2Δn** per la somma di operandi da n bit, se Δ è il ritardo di porta)

- Nasce la necessità di sviluppare nuove architetture che presentino tempi di propagazione inferiori:
  - Potendo disporre in anticipo di tutti i riporti la somma sui vari bit potrebbe essere calcolata in parallelo

Condizioni di generazione e propagazione dei riporti

Si introducono due funzioni booleane ausiliarie:

- $P_i = x_i + y_i$ condizione di *propagazione*
- $G^i = x_i y_i$ condizione di *generazione*

Il riporto in uscita dallo stadio i-esimo assume la forma:

$c_{i+1} = x_i y_i + (x_i + y_i) c_i = G_i + P_i\, c_i$ dove ci può essere scritto come:

$c_i = x_{i-1} y_{i-1} + (x_{i-1} + y_{i-1}) c_{i-1} = G_{i-1} + P_{i-1}\, c_{i-1}$ da cui:

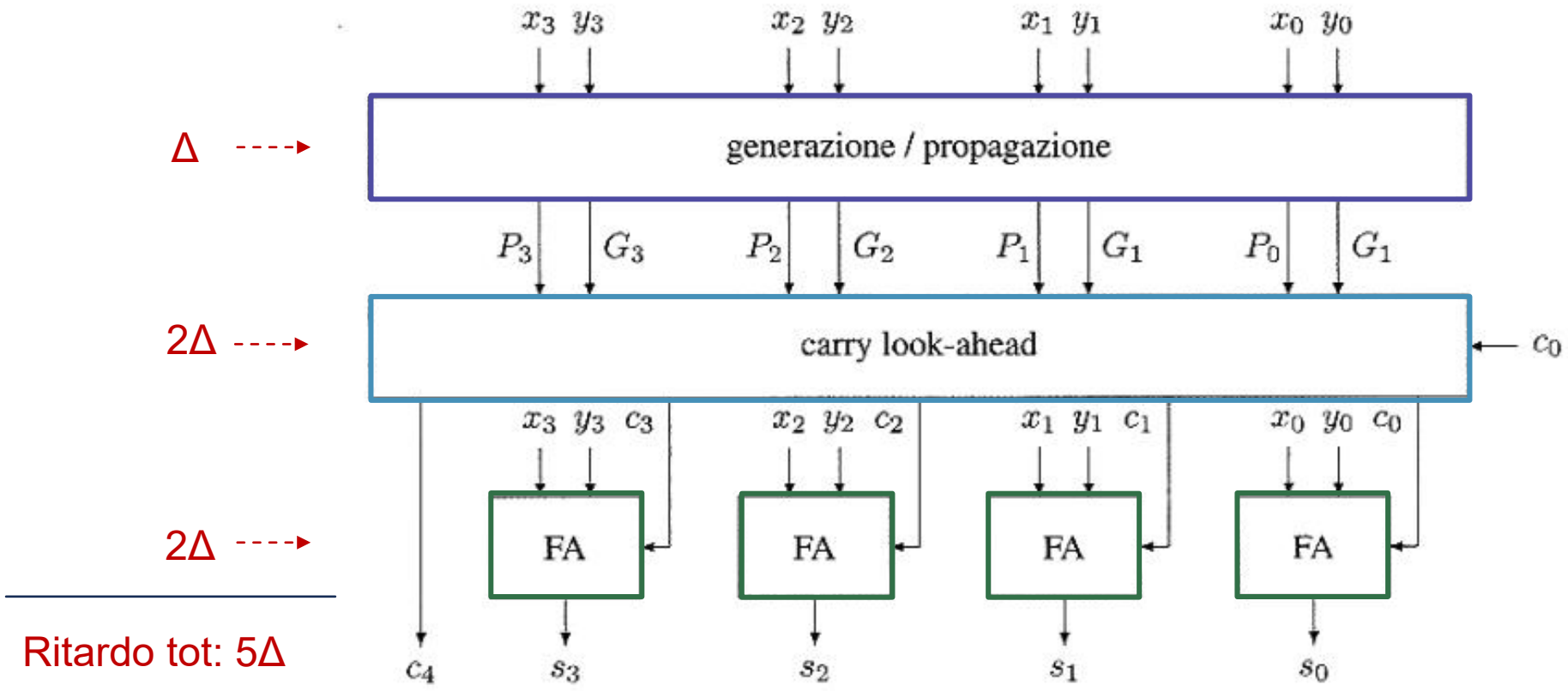
$$\boxed{c_{i+1} = G_i + P_i c_i = G_i + P_i\,(G_{i-1} + P_{i-1}\, c_{i-1}) = G_i + P_i G_{i-1} + P_i\, P_{i-1}\, c_{i-1}}$$

Condizioni di generazione e propagazione dei riporti

Ripetendo lo stesso procedimento è possibile esprimere ciascun riporto in funzione di c_0:

$$
\begin{aligned}
c_1 &= G_0 + P_0 c_0 \\
c_2 &= G_1 + P_1 G_0 + P_1 P_0 c_0 \\
c_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0 \\
c_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0 \\
&\ldots
\end{aligned}
$$

Sommatore Carry Look-ahead



Ritardo tot: 5Δ

- Il ritardo è pari a 5 Δ
- Si puo' dimostrare che l'area occupata è pari a (n2 + 9*n)/2

Sommatore Carry Look-ahead: ottimizzazione

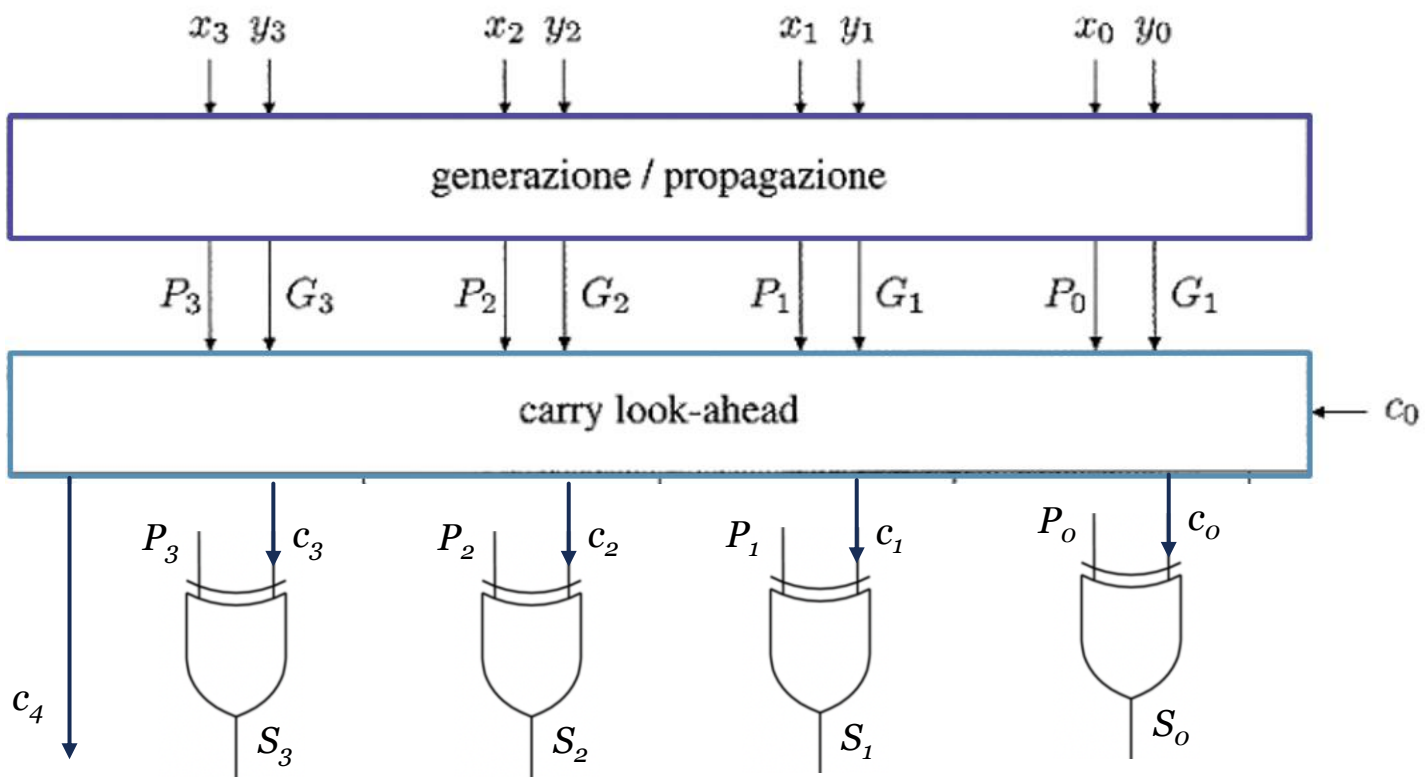
Ricordiamo che: $S = x \oplus y \oplus Cin = P \oplus Cin$

Δ

2Δ

Δ

Ritardo tot: 4Δ



OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORI VELOCI

Sommatore Carry-Select

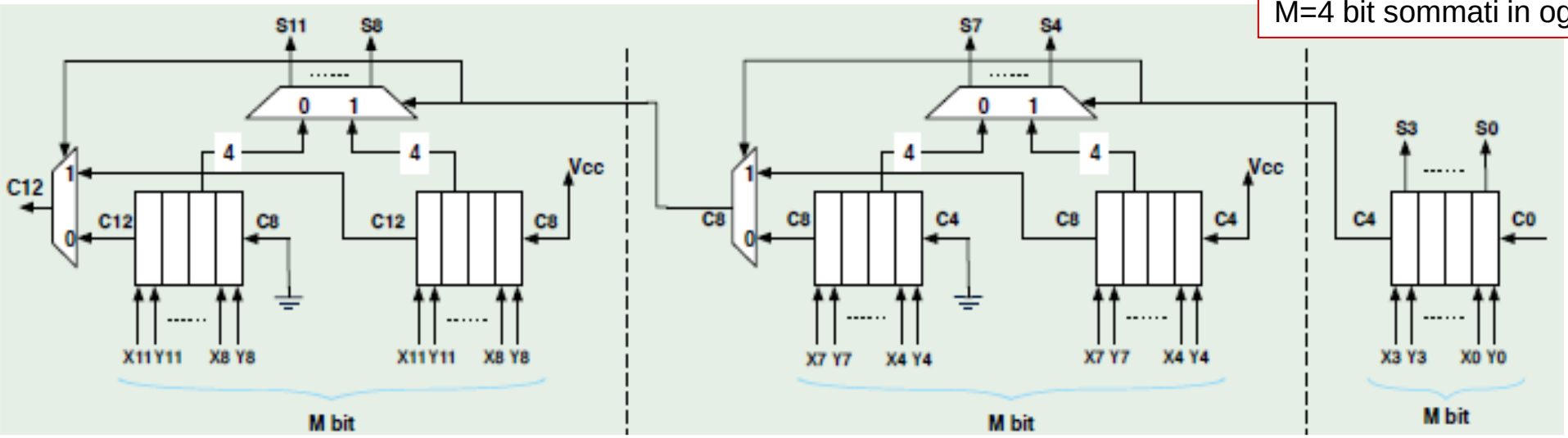
Il problema dell'architettura RCA è che lunghezza della carry chain aumenta linearmente con l'aumentare del numero di bit.

- Si può pensare di suddividere tale catena in catene più corte.

Supponiamo di dividere un normale RCA in P blocchi, ciascuno dei quali somma M bit:

- il primo blocco è un RCA di M bit che restituisce in uscita la somma su M bit ed il riporto in uscita (overflow);
- i blocchi successivi contengono 2 RCA: entrambi effettuano la somma degli stessi M bit ma uno ha Cin = 0, l'altro Cin = 1.

P=3 blocchi
M=4 bit sommati in ogni blocco



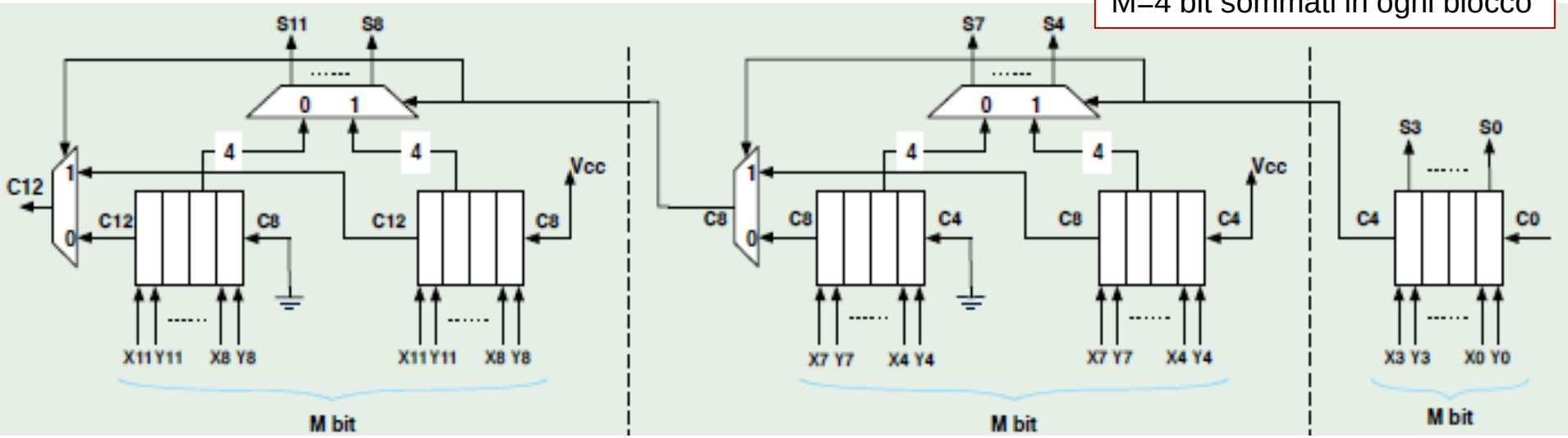
Sommatore Carry-Select

Definiamo TFA e TMUX i tempi di propagazione rispettivamente del Full Adder e del Multiplexer:

- Il CM, ovvero il carry in uscita dal primo blocco, è disponibile al tempo M*TFA;
- Le somme ed il carry all'uscita del secondo blocco saranno disponibili al tempo M* TFA + TMUX;
- Le somme ed il carry finali saranno disponibili al tempo:

$$T = M * TFA + (P-1) * TMUX$$

P=3 blocchi
M=4 bit sommati in ogni blocco



OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMA DI PIÙ OPERANDI: SOMMATORI CARRY SAVE

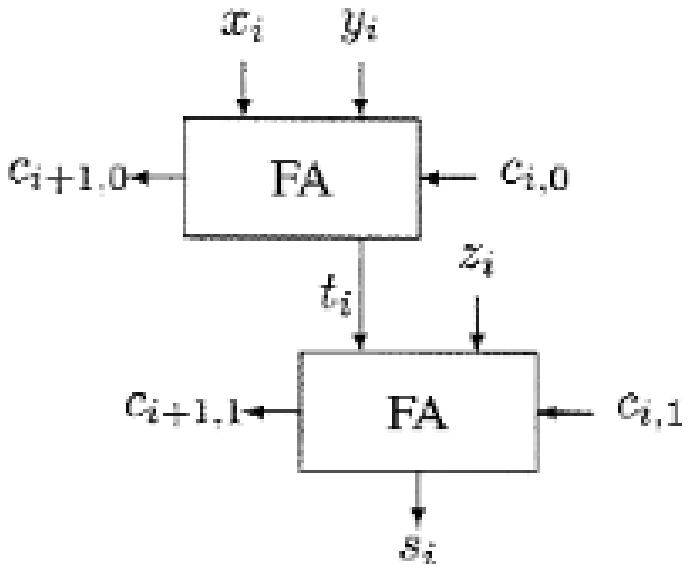
Somma di operandi multipli

Si consideri la somma S = X+Y+Z

Utilizzando **due sommatori in cascata** si può realizzare tale operazione come:

$$S = (X+Y) + Z = T + Z$$

Considerando un'architettura composta da due sommatori ripple-carry la somma relativa al bit i-esimo può essere effettuata con il seguente schema:



Secondo questo schema si ricava che:

$$t_i = x_i + y_i + c_{i,0}$$

$$s_i = t_i + z_i + c_{i,1}$$

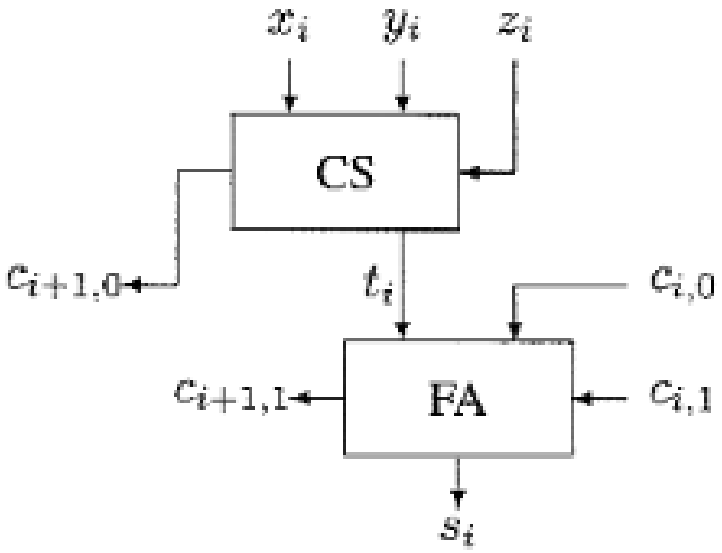
Da cui:

$$\boxed{s_i = (x_i + y_i + z_i) + c_{i,0} + c_{i,1}}$$

Sommatori carry-save

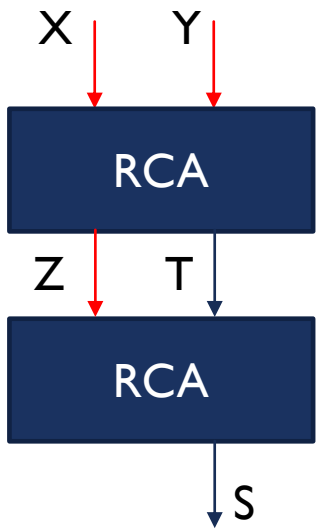
$$s_i = (x_i + y_i + z_i) + c_{i,0} + c_{i,1}$$

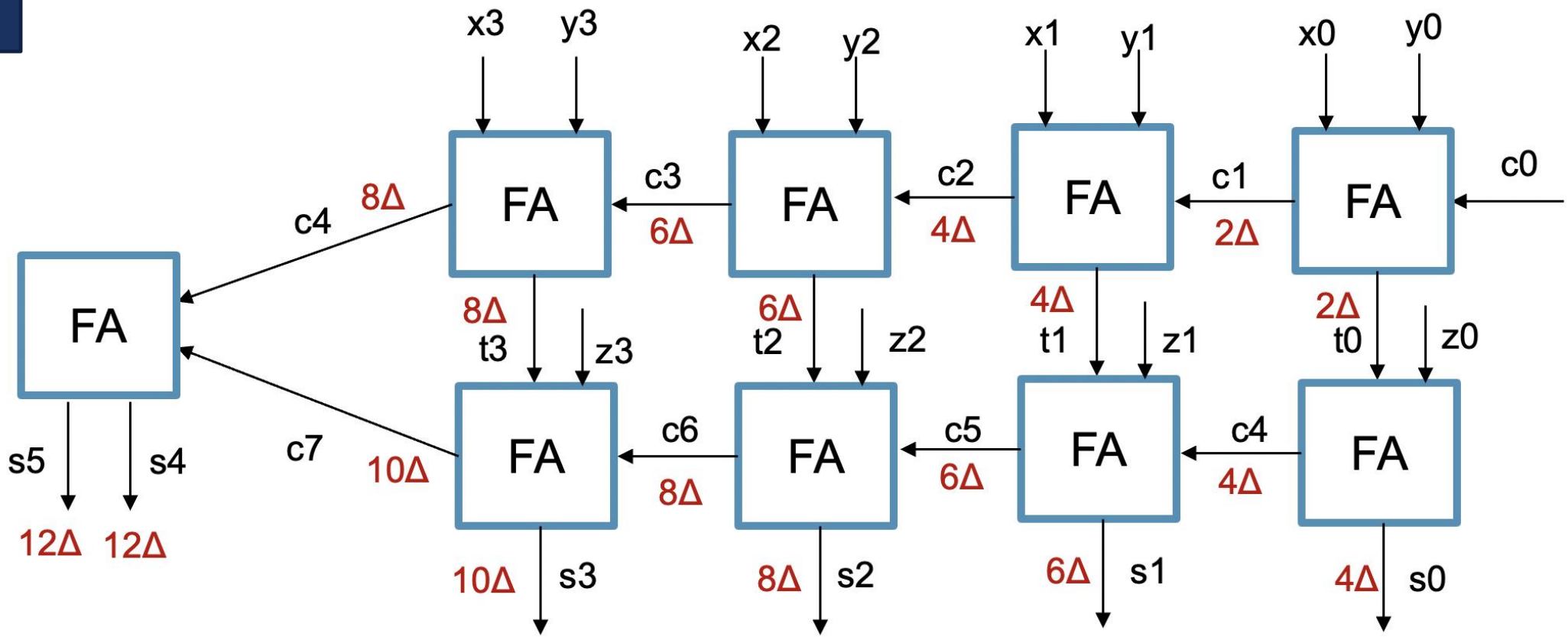
La somma del bit i-esimo dei 3 operandi può essere effettuata direttamente da un full adder (indicato con il simbolo CS)
Dalla rete dei CS escono 2 stringhe, una di «somma» e una di «riporto», che devono essere poi sommate fra loro tramite un sommatore ripple carry



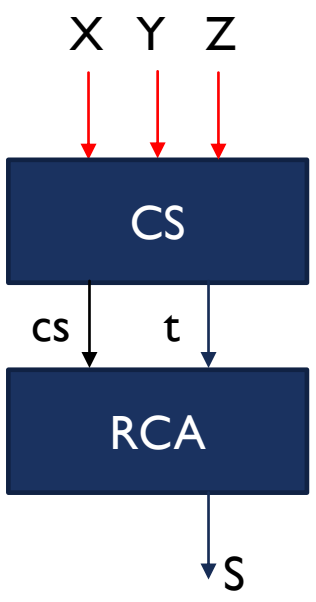
Utilizzando il modulo di base mostrato si possono costruire sommatori carry save con operandi di dimensione arbitraria: **i blocchi CS operano in parallelo** in quanto non sono soggetti alla propagazione del riporto mentre i blocchi FA sono connessi a formare un sommatore ripple-carry

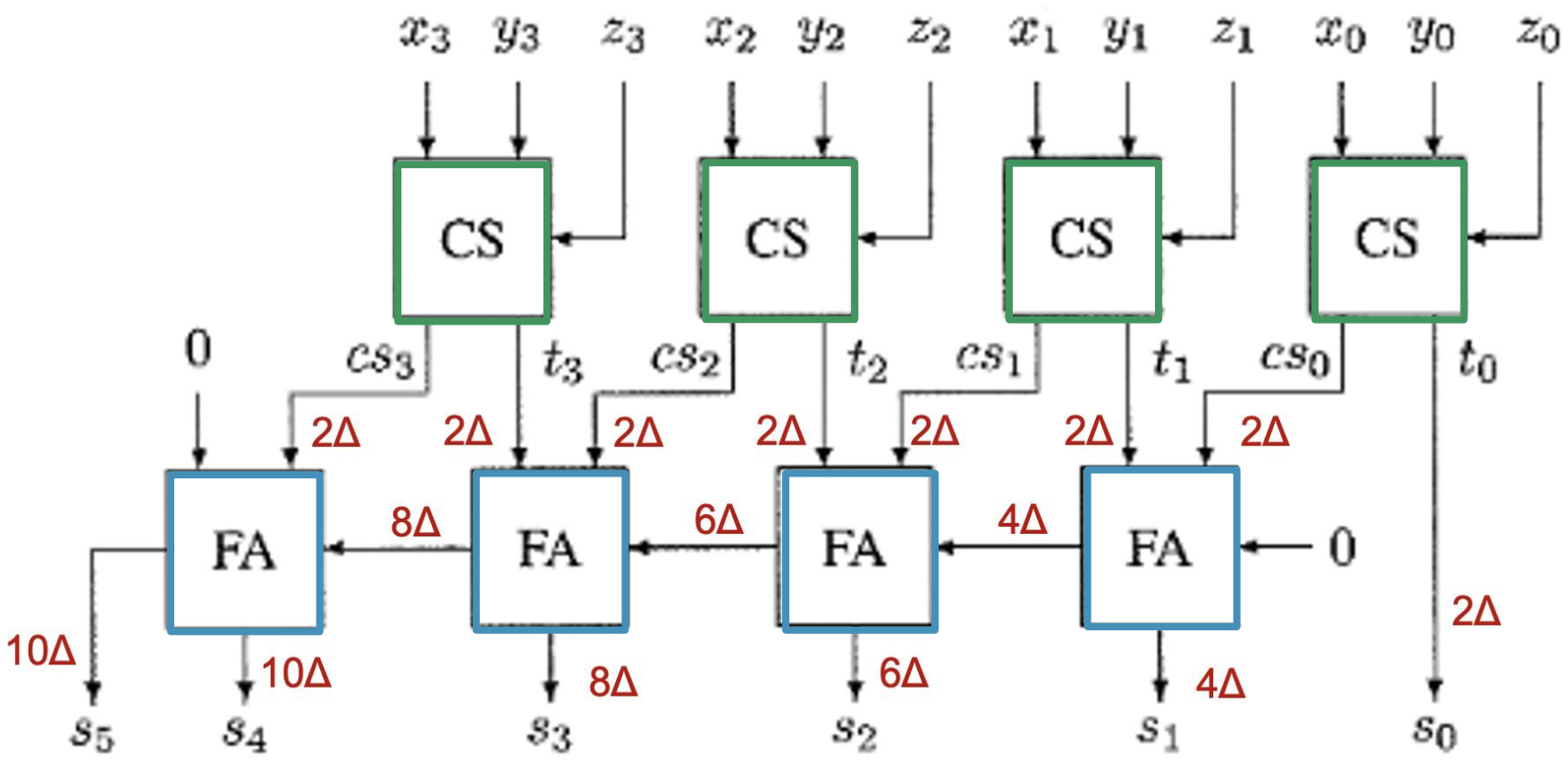
Esempio: Somma di 3 stringhe con RCA





Sempio: Somma di 3 stringhe con carry-save





Implementazione strutturale VHDL del carry-save

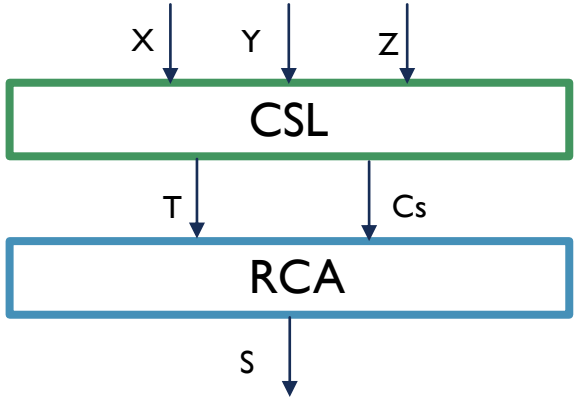
```vhdl
--sommatore carry save di 3 stringhe da N bit
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity carry_save is
        generic(N: natural:=8);
        port(   X, Y, Z: in std_logic_vector(N-1 downto 0);
                         s: out std_logic_vector(N+1 downto 0));
end carry_save;
architecture structural of carry_save is
        component full_adder is
                port(   a,b,cin: in std_logic;
                                r,cout: out std_logic);
        end component;
        signal cs: std_logic_vector(N-1 downto 0);
        signal t,temp: std_logic_vector(N downto 0);
        begin
        cs0ton1: for i in 0 to N-1 generate
                csi: full_adder port map(X(i),Y(i),Z(i),t(i),cs(i));
        end generate;
        t(N)<='0';
        temp(0)<='0';
        fa0ton1: for i in 0 to N-1 generate
                fa: full_adder port map(t(i+1),cs(i),temp(i),s(i+1),temp(i+1));
        end generate;
```

Il componente di base è il full-adder

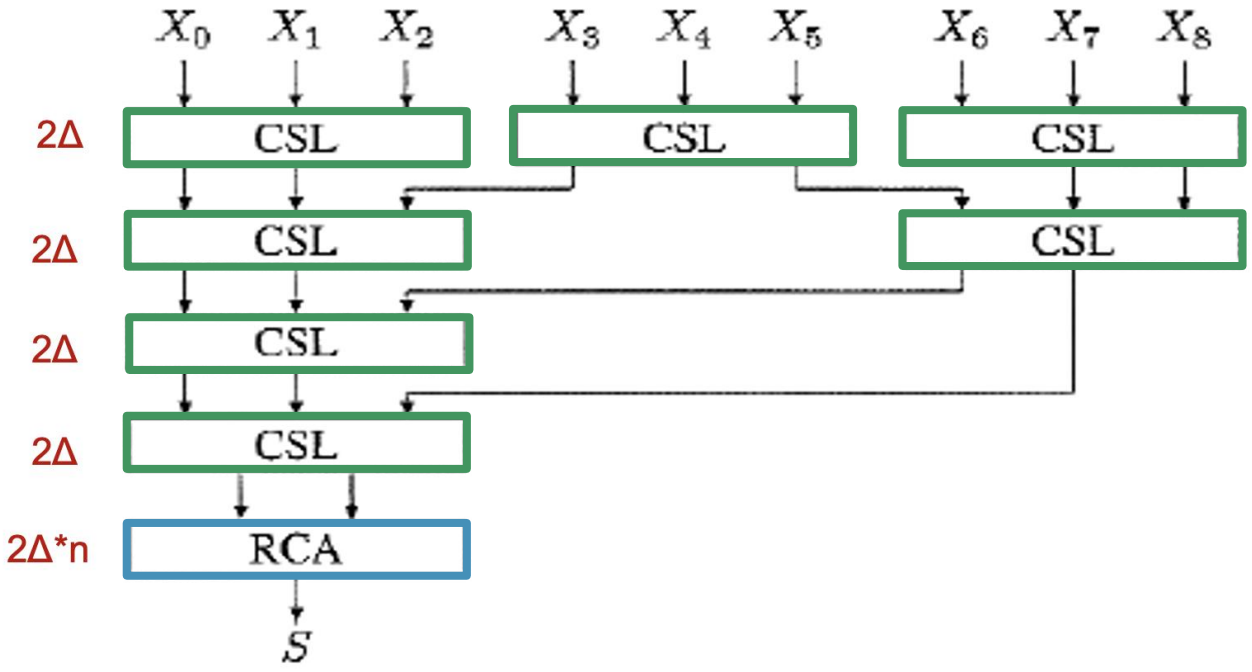
Carry-save logic: non c'è propagazione sulla riga

RCA: il riporto (cs) ha un peso maggiore della somma parziale (t)

Sommatori carry-save: somme di n stringhe



Un sommatore carry-save è costituito da un blocco CSL o carry-save logic e da un blocco ripple carry adder. Interconnettendo opportunamente blocchi carry-save è possibile realizzare la somma di un numero elevato di operandi riducendo il ritardo complessivo



OPERAZIONI ARITMETICHE SU NUMERI INTERI

SOMMATORE SEQUENZIALE

Approccio sequenziale

Gli addizionatori visti sono tutte **macchine combinatorie**

In alternativa, si può adottare un approccio ***sequenziale***: invece di replicare l'operazione nello spazio (usando n FA) replichiamo l'operazione nel tempo.

Studiamo l'addizione di stringhe di bit come un algoritmo iterativo, costituito da più passi.

Serve allora realizzare l'architettura in termini di:

- **Unità operativa**
- **Unità di controllo**

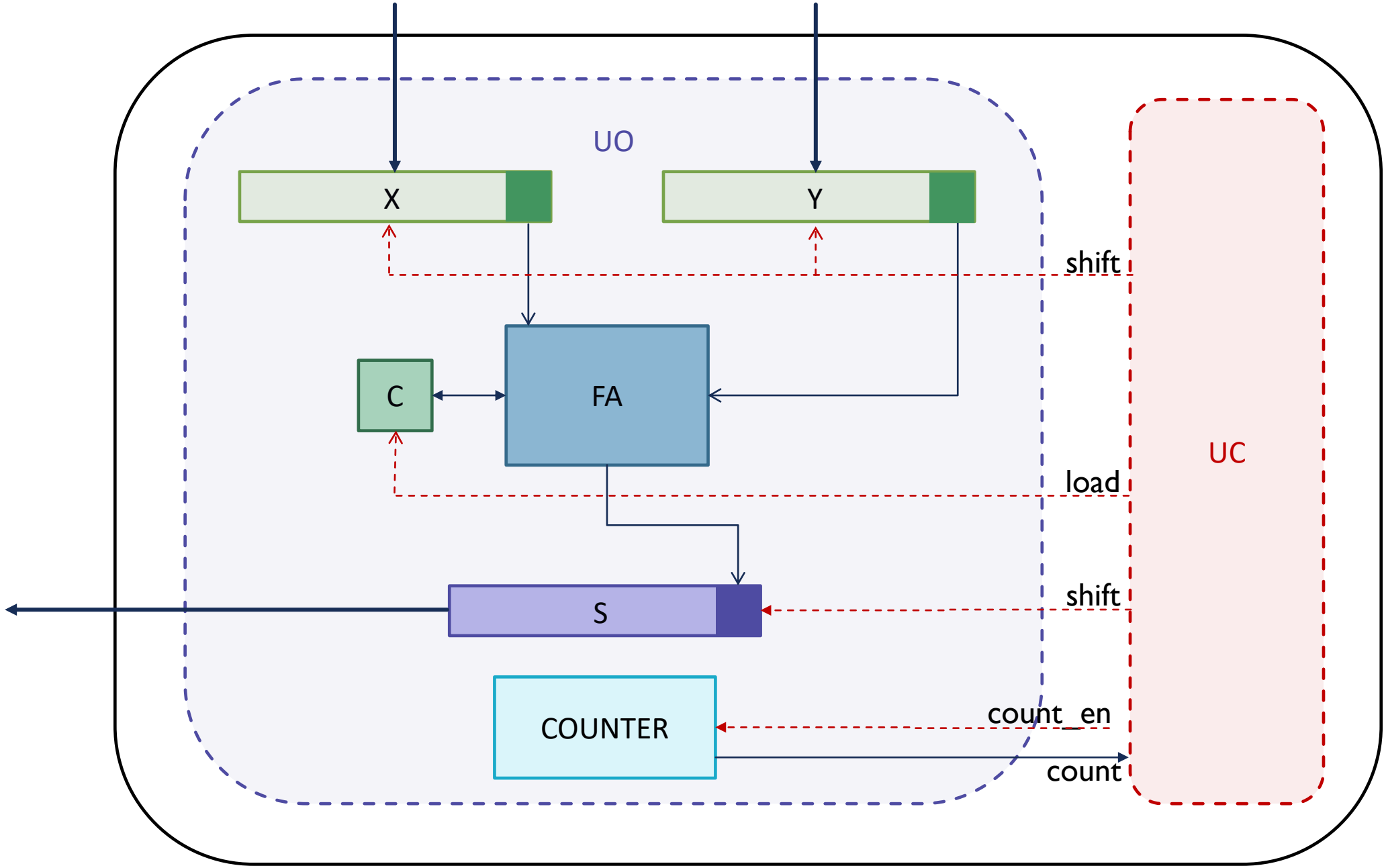
Approccio sequenziale

Al passo i:

- Facciamo la somma di Xi e Yi considerando l'eventuale riporto generato allo stadio i-1 (Ci), generiamo Si e memorizziamo Ci+1, per il passo successivo } ***ADD***

- Carichiamo gli operandi per il passo i+1 → ***SHIFT***

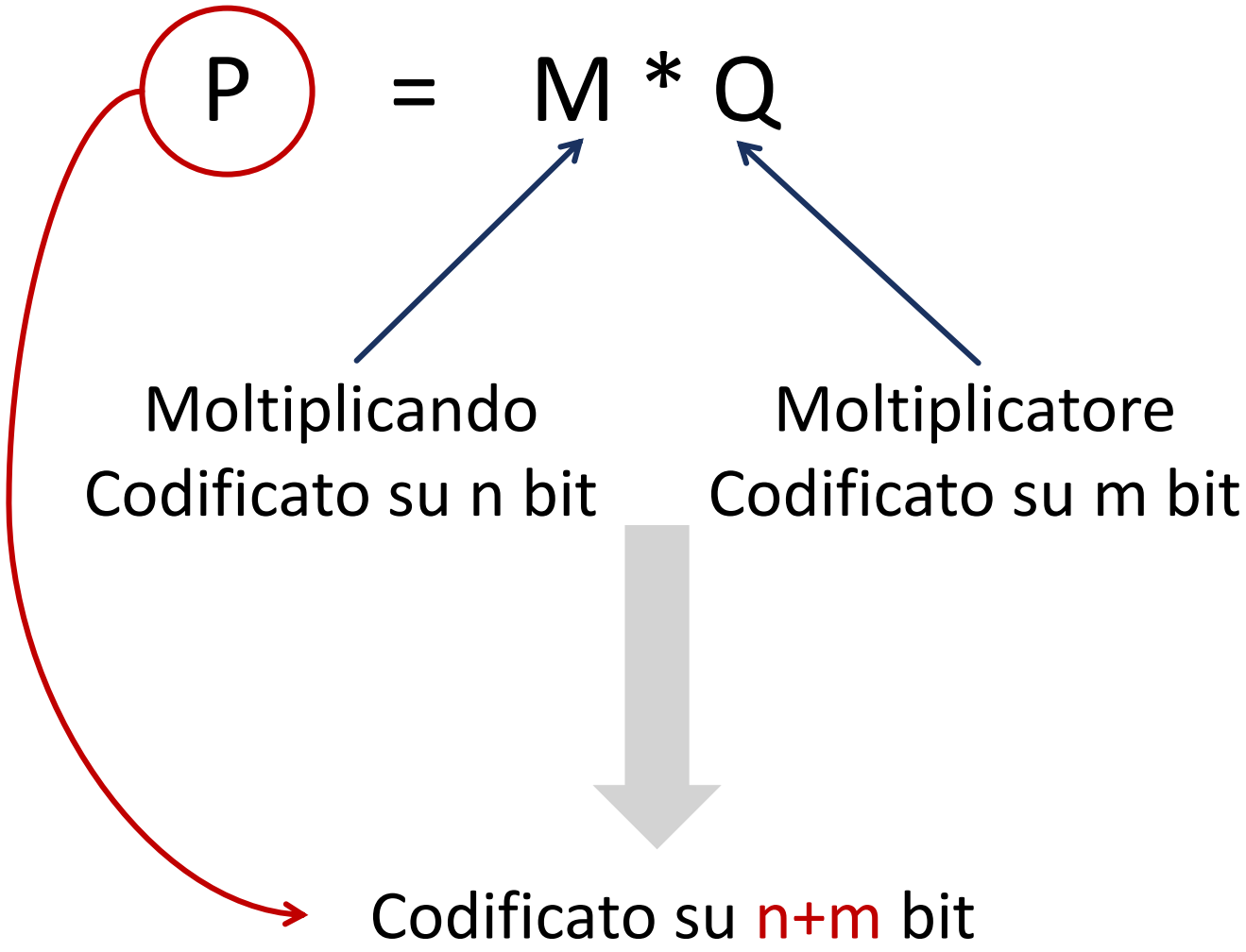
Approccio sequenziale: potenziale architettura



OPERAZIONI ARITMETICHE SU NUMERI INTERI

MOLTIPLICATORI

Parallelismo degli operandi



Moltiplicazione interi senza segno

- La moltiplicazione di numeri interi *senza segno* si esegue con lo stesso metodo usato per la moltiplicazione decimale.
- Supponiamo di voler calcolare: P = A * B con **A** = 13; **B** = 11

Moltiplicazione decimale

```
        1   3   x
        1   1   =
    ---------------
        1   3   +
    1   3   -   =
    ---------------
    1   4   3
```

Moltiplicazione binaria

```
                1 1 0 1   x
                1 0 1 1   =
    ---------------------------
                0 0 0 0 1 1 0 1   +
Matrice dei prodotti  0 0 0 1 1 0 1 -   +
parziali              0 0 0 0 0 0 -     +
                      0 1 1 0 1 -       =
    ---------------------------
                1 0 0 0 1 1 1 1
```

Tipologia di soluzioni

- Moltiplicatori ***paralleli***
  - Prodotto delle cifre + somma di righe.
  - Multiply and accumulate.
- Moltiplicatori ***seriali***
  - Derivati da procedura manuale (Roberston).
  - Basati su codifiche alternative (Booth).

OPERAZIONI ARITMETICHE SU NUMERI INTERI

MOLTIPLICATORI PARALLELI

Moltiplicazione a due fasi

Possiamo allora distinguere due fasi nella moltiplicazione:

- **Prima fase:** determinazione della matrice dei prodotti parziali.
  - Viene effettuata semplicemente utilizzando opportune porte AND.
- **Seconda fase**: somma dei prodotti parziali.
  - Può essere effettuata utilizzando varie tecniche con l'obiettivo di aumentare l'efficienza (velocità) del circuito → è ciò su cui differiscono le architetture che studiamo.

Determinazione matrice per il prodotto di fattori positivi

Dato il moltiplicando A (che per semplicità supponiamo rappresentato su 4 bit) ed il moltiplicatore B (anch'esso su 4 bit) in rappresentazione binaria senza segno:

$$A = a_3 * 2^3 + a_2 * 2^2 + a_1 * 2^1 + a_0 * 2^0$$

$$B = b_3 * 2^3 + b_2 * 2^2 + b_1 * 2^1 + b_0 * 2^0$$

Il prodotto P = A* B è esprimibile come:

$$P = (a_3 * 2^3 + a_2 * 2^2 + a_1 * 2^1 + a_0 * 2^0) * (b_3 * 2^3 + b_2 * 2^2 + b_1 * 2^1 + b * 2^0) =$$

$$= a_3 b_3 * 2^6 + (a_2\ b_3 + a_3\ b_2) * 2^5 + (a_1\ b_3 + a_2\ b_2 + a_3\ b_1) * 2^4 +$$

$$+ (a_0\ b_3 + a_1\ b_2 + a_2\ b_1 + a_3\ b_0) * 2^3 + (a_0\ b_2 + a_1\ b_1 + a_2\ b_0) * 2^2 +$$

$$+ (a_0\ b_1 + a_1\ b_0) * 2^1 + a_0 b_0 * 2^0$$

Ogni termine $a_i b_j$ è il ***prodotto aritmetico*** di due cifre, una del moltiplicando A e una del moltiplicatore B, ed ha peso 2^{i+j}.

Fattori di un prodotto ordinati

Rappresentiamo i bit di prodotto parziale su una **matrice iniziale diagonale**, incolonnati in funzione del loro peso.

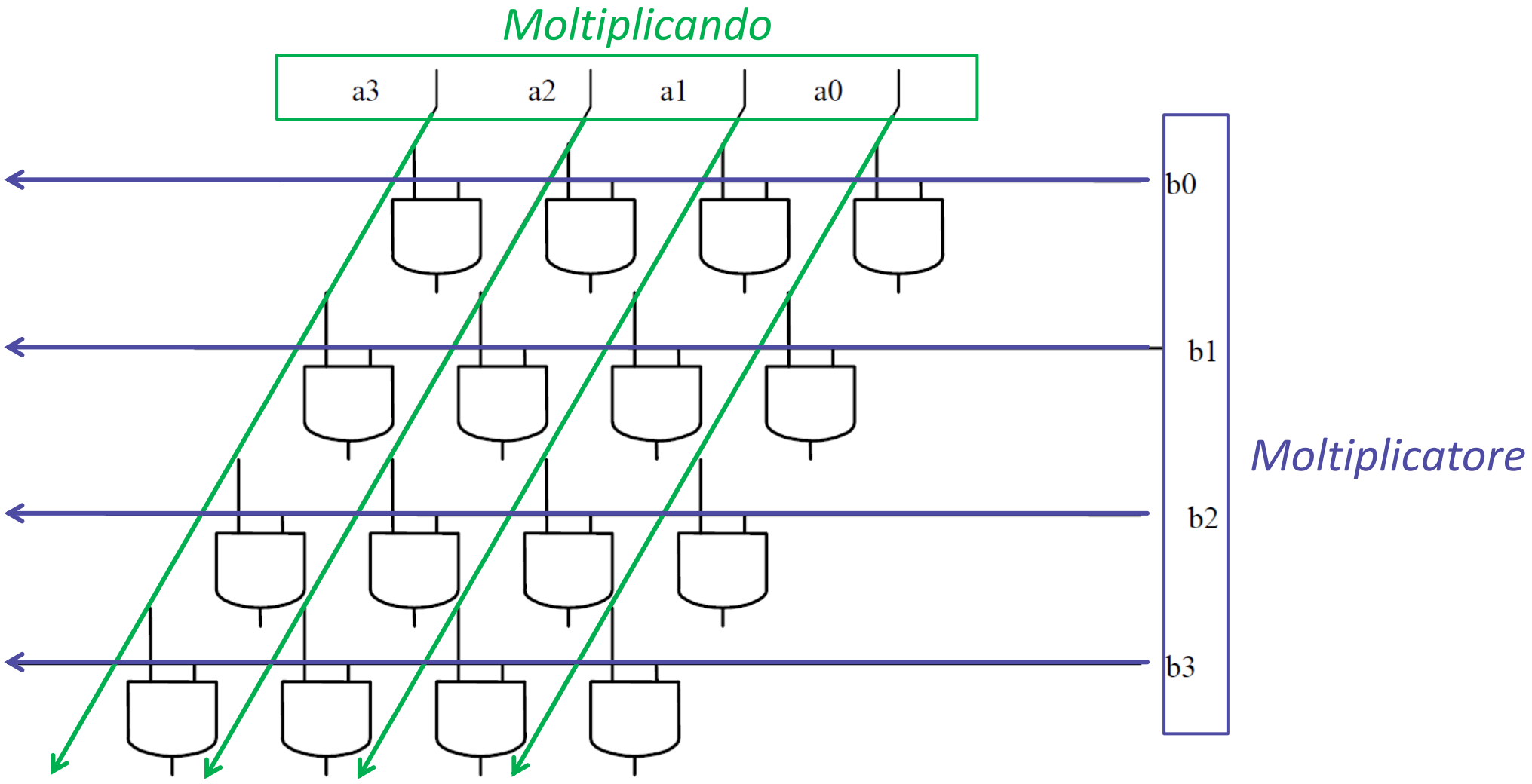
Moltiplicando

			a_3	a_2	a_1	a_0	
			a_3b_0	a_2b_0	a_1b_0	a_0b_0	b_0
		a_3b_1	a_2b_1	a_1b_1	a_0b_1		b_1
	a_3b_2	a_2b_2	a_1b_2	a_0b_2			b_2
a_3b_3	a_2b_3	a_1b_3	a_0b_3				b_3

Moltiplicatore

Dato che ogni cifra può assumere solo i valori 0/1, il prodotto aritmetico $a_i b_j$ coincide con il ***prodotto logico*** dei due bit a_i e b_j → **AND**

Circuito che effettua i prodotti



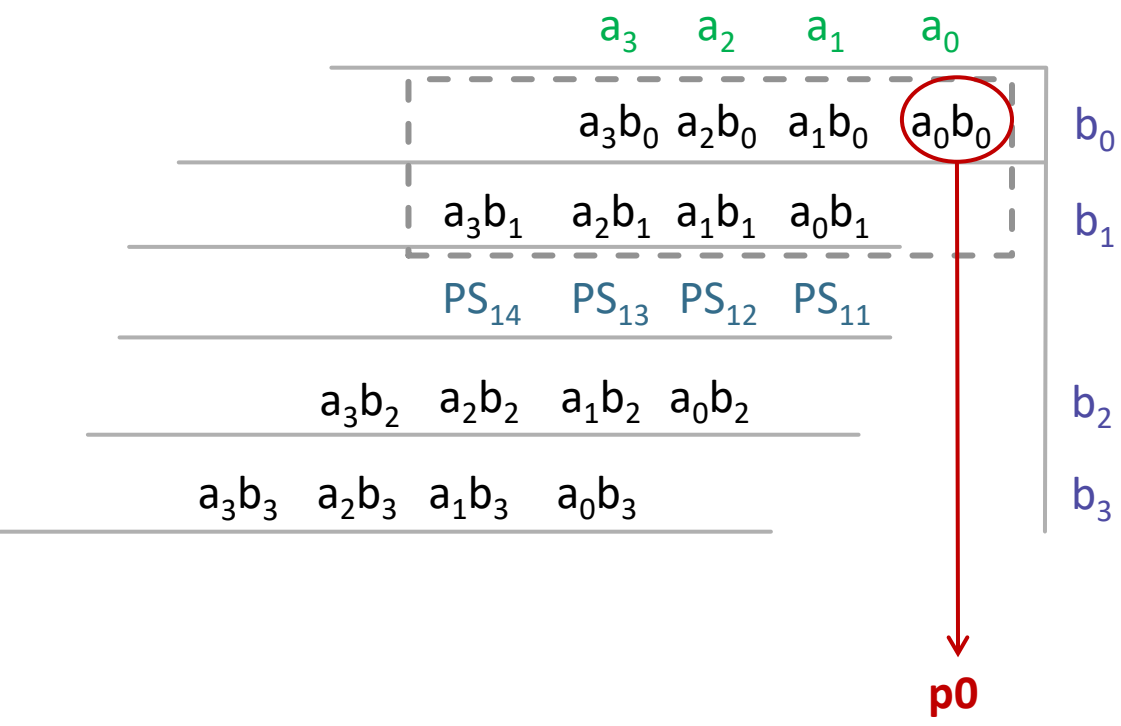
PRODOTTO COME SOMMA PER RIGHE (RIPPLE CARRY MULTIPLICATION)

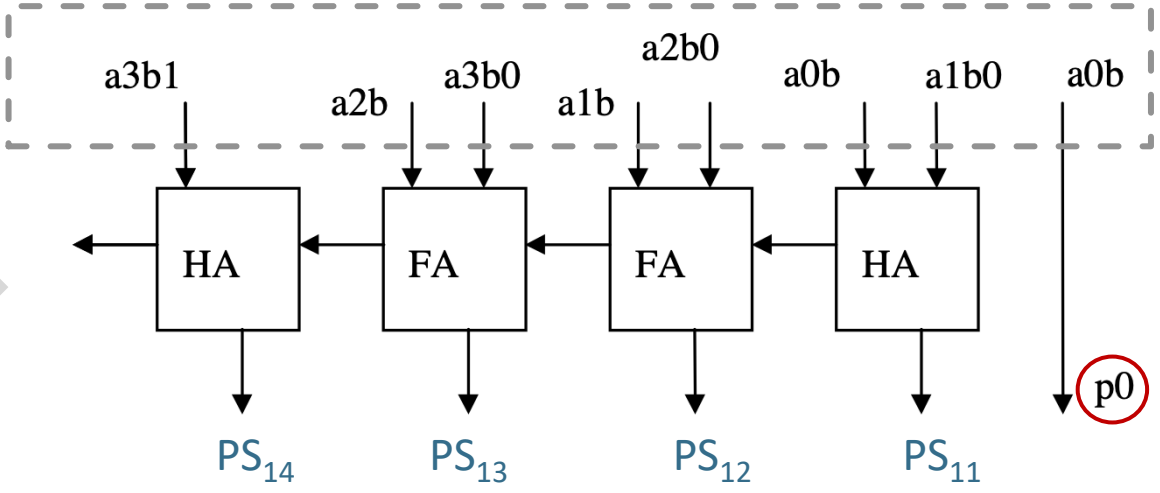
- Realizzata la prima fase dell'operazione prodotto (determinazione della matrice diagonale ed il circuito di porte AND che la realizza), la *seconda fase* calcola il prodotto come somma per righe.
  - Le n righe della matrice dei prodotti parziali vengono sommate utilizzando **n-1 sommatori*** (i posti vuoti sono degli "zeri").
- La struttura ottenuta è molto regolare e, quindi, ben integrabile.

* Per sommatori intendiamo il Ripple Carry Adder, NON la singola cella half-adder/full-adder.

Circuito di base per la somma di due righe

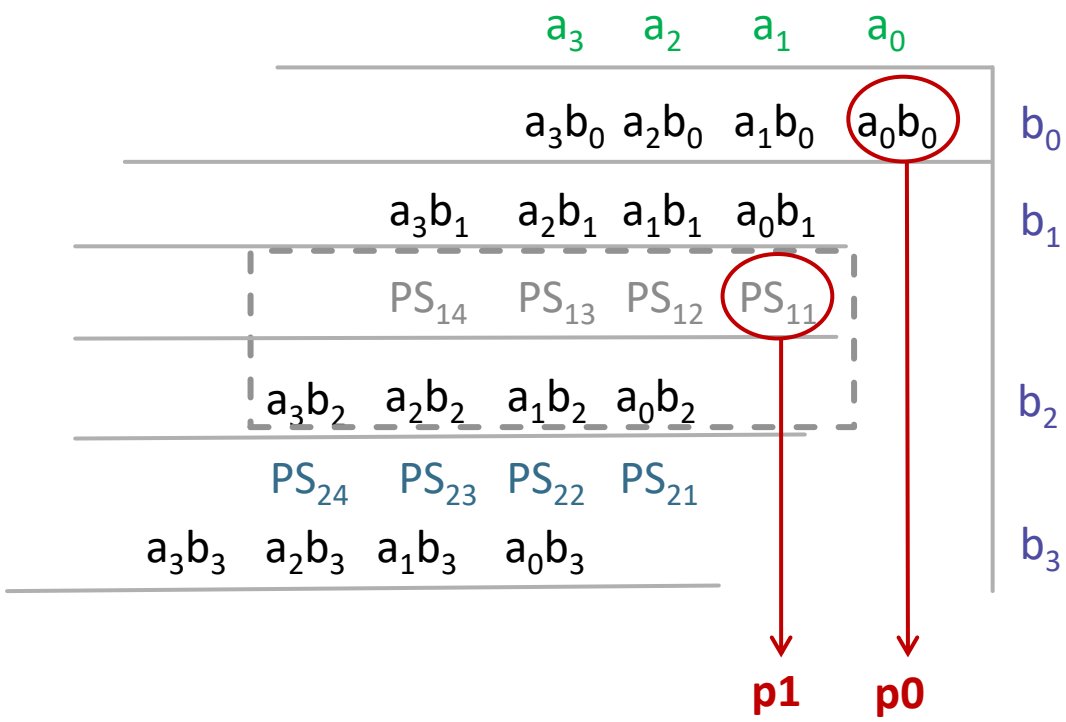
- Sommiamo le *prime due righe* della matrice dei prodotti parziali con due HA e due FA (il primo e l'ultimo stadio sono half-adder in quanto ricevono solo due entrate).

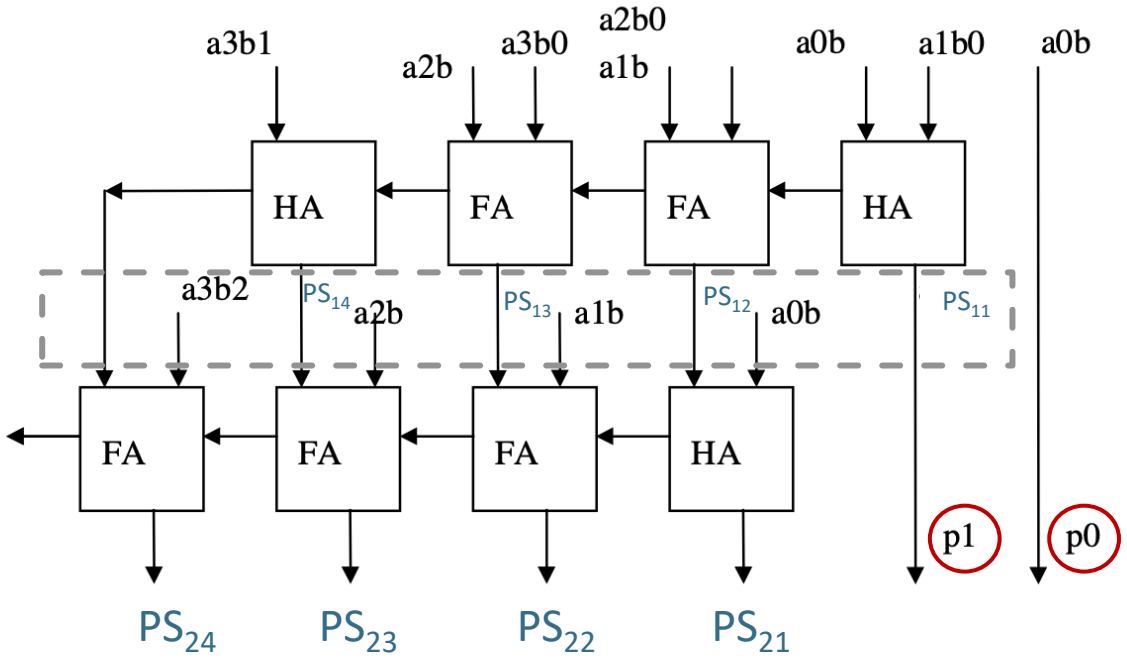




Aggiunta della terza riga

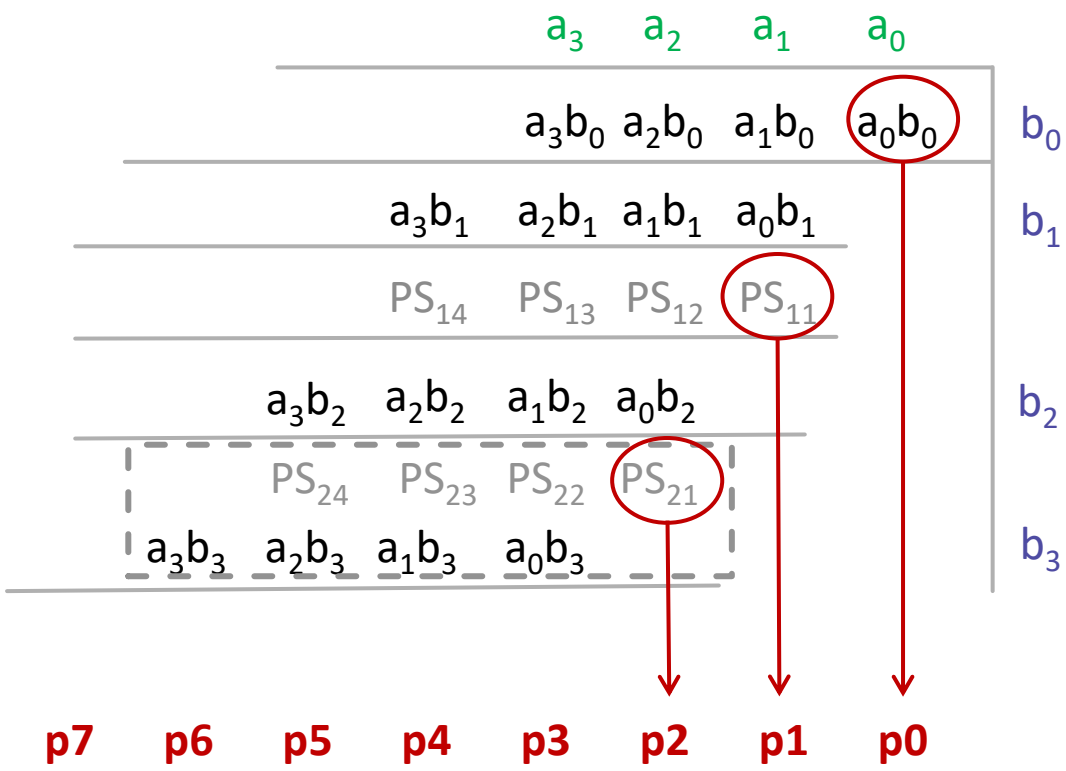
- L'uscita del primo addizionatore [PS_{11}: PS_{14}] deve essere sommata alla terza riga.

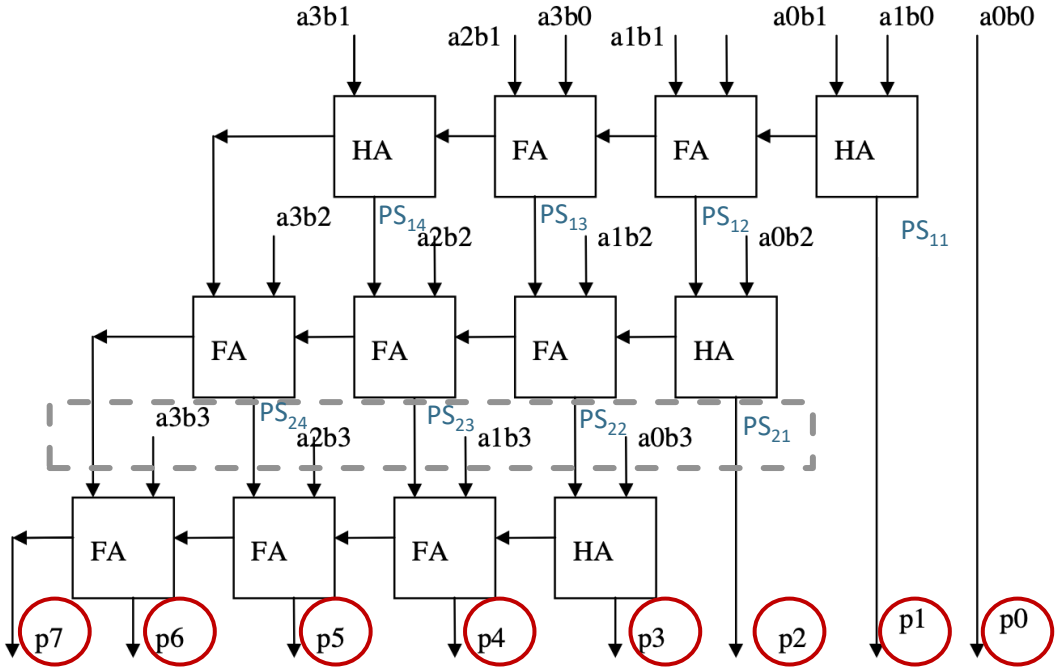




Circuito completo per la somma di quattro righe

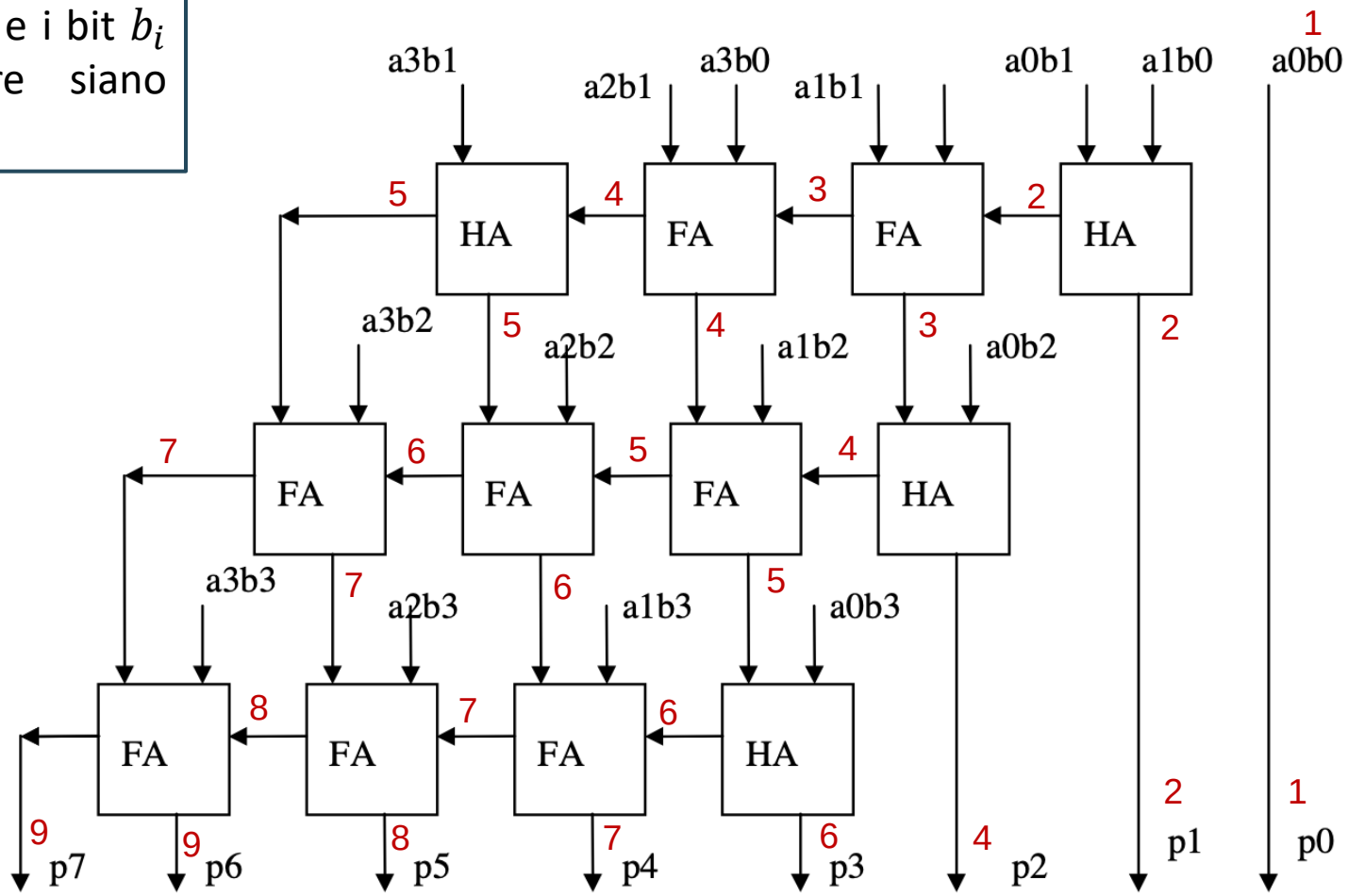
- L'uscita del secondo addizionatore [PS_{21}: PS_{24}] deve essere sommata alla quarta ed ultima riga.





Calcolo dei tempi di computazione dell'RCM

Supponiamo che i bit a_i del moltiplicando e i bit b_i del moltiplicatore siano noti all'istante 0.



Tutti gli AND sono attivi simultaneamente → i 16 bit $a_i * b_i$ risultano noti all'istante 1.

Riassumendo…

- Supponendo che $1T$ sia il tempo di computazione di un FA/HA per calcolare bit di somma e bit di riporto, considerando la disponibilità all'istante 0 degli operandi:
  - I prodotti parziali sono disponibili dopo un quanto di tempo T.
  - Per ogni FA, detti t_a, t_b, t_{cin} gli istanti in cui sono noti i tre segnali di ingresso (ad un FA), $\boldsymbol{t_{cout;s}}$ è l'istante di tempo in cui sono disponibili le due uscite del FA e si calcola come:

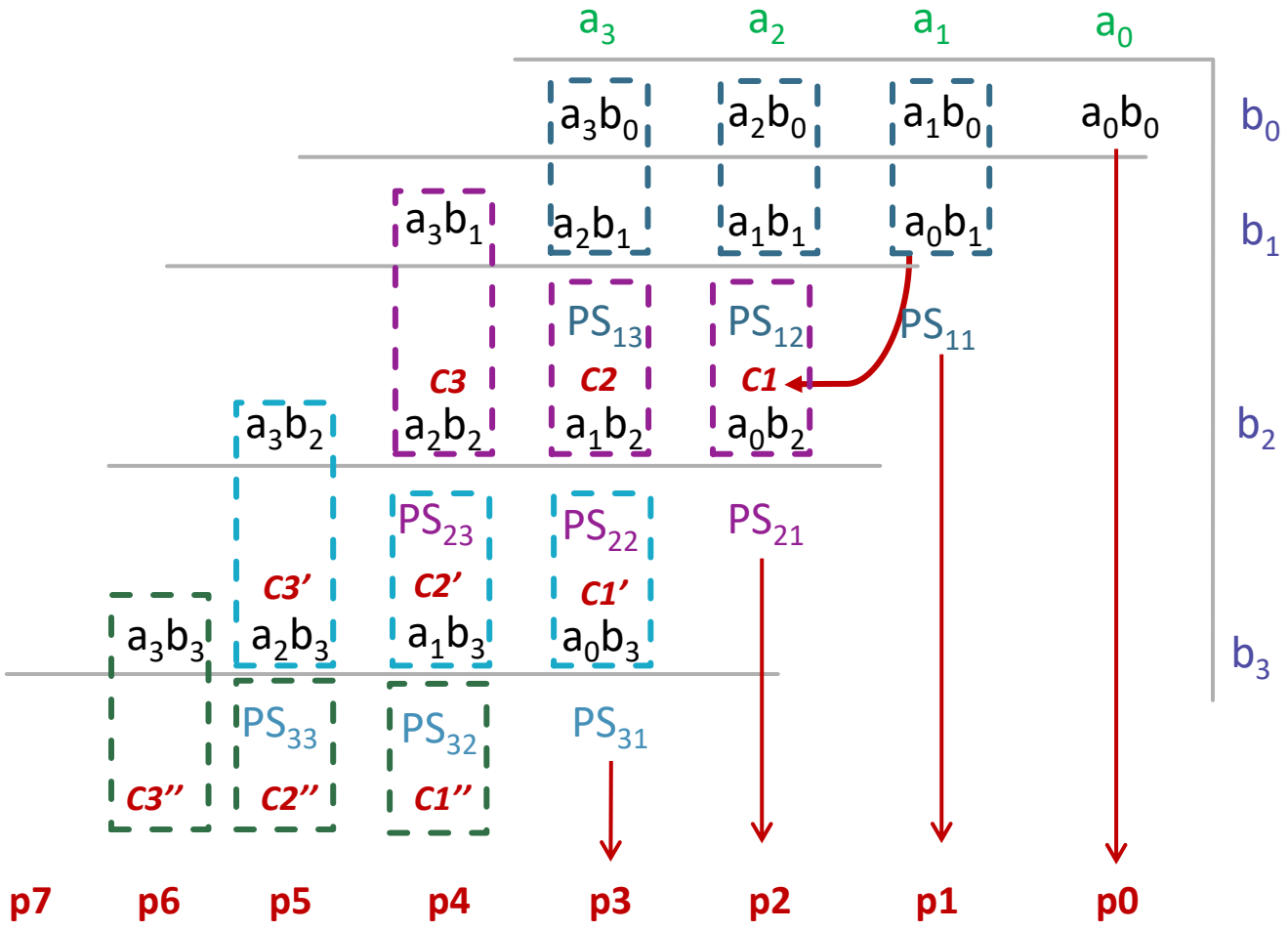
$$t_{cout;s} = \max(t_a, t_b, t_{cin}) + 1$$

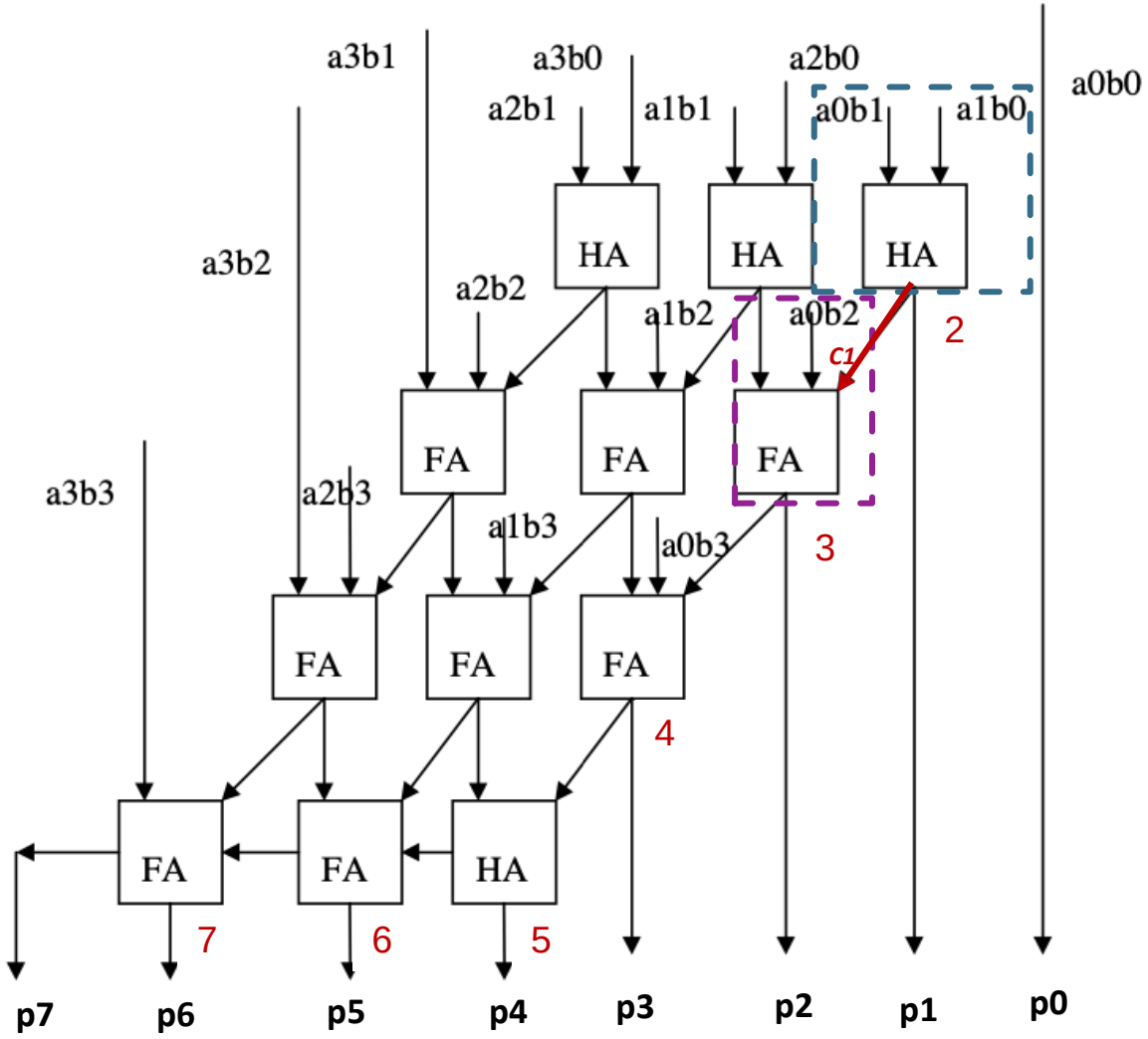
- Dato che i riporti si propagano tutti nella stessa direzione, i tempi richiesti per la moltiplicazione dipendono *linearmente* dal numero di bit dei fattori (i ritardi sono quindi circa doppi di quelli necessari per la somma).

PRODOTTO COME SOMMA PER DIAGONALI

- Realizzata la prima fase dell'operazione prodotto (determinazione della matrice diagonale ed il circuito di porte AND che la realizza), la *seconda fase* calcola il prodotto come somma per diagonali.
  - La propagazione del riporto non avviene più sulla riga → si ha una propagazione del **riporto in diagonale**. Il numero di sommatori necessari è **n** (uguale al numero di righe della matrice di prodotti parziali).
- La struttura ottenuta è ancora una volta regolare e quindi ben integrabile.

Prodotto come somma per diagonali





CONFRONTO FRA LE DUE ARCHITETTURE

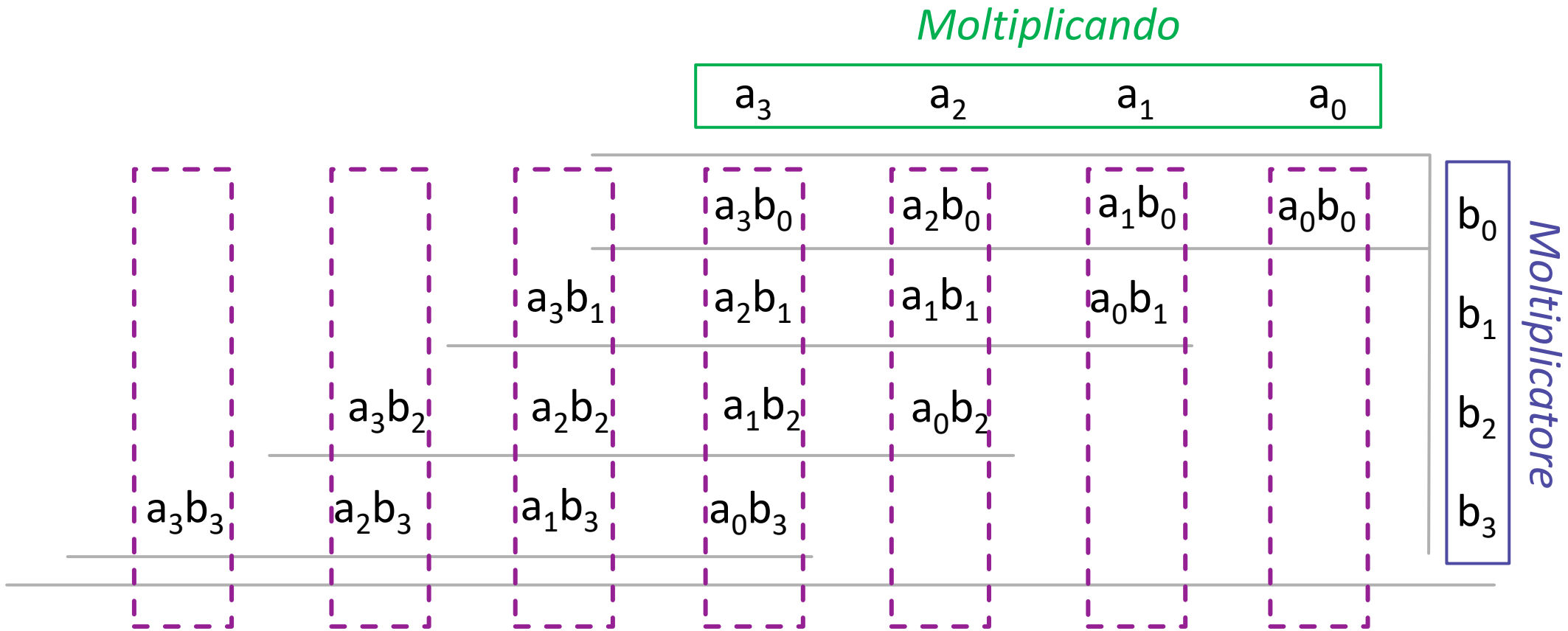
- Entrambe le architetture sono facilmente integrabili su silicio, sono fortemente regolari e ripetitive: l'RCM è caratterizzato da 3 righe di 4 HA/FA, il prodotto come somma per diagonali è composto da 4 righe di 3 HA/FA.
- Il numero di HA e FA è uguale → il costo è uguale.
- Il circuito che esegue la somma per diagonali è più veloce di quello che esegue la somma per righe: il percorso critico del primo è 7; quello del secondo è 9.

PRODOTTO COME SOMMA PER COLONNE

- Realizzata la prima fase dell'operazione prodotto (determinazione della matrice diagonale ed il circuito di porte AND che la realizza), la *seconda fase* calcola il prodotto come somma per colonne.
  - Il prodotto viene calcolato sommando le **colonne della matrice** dei prodotti parziali.

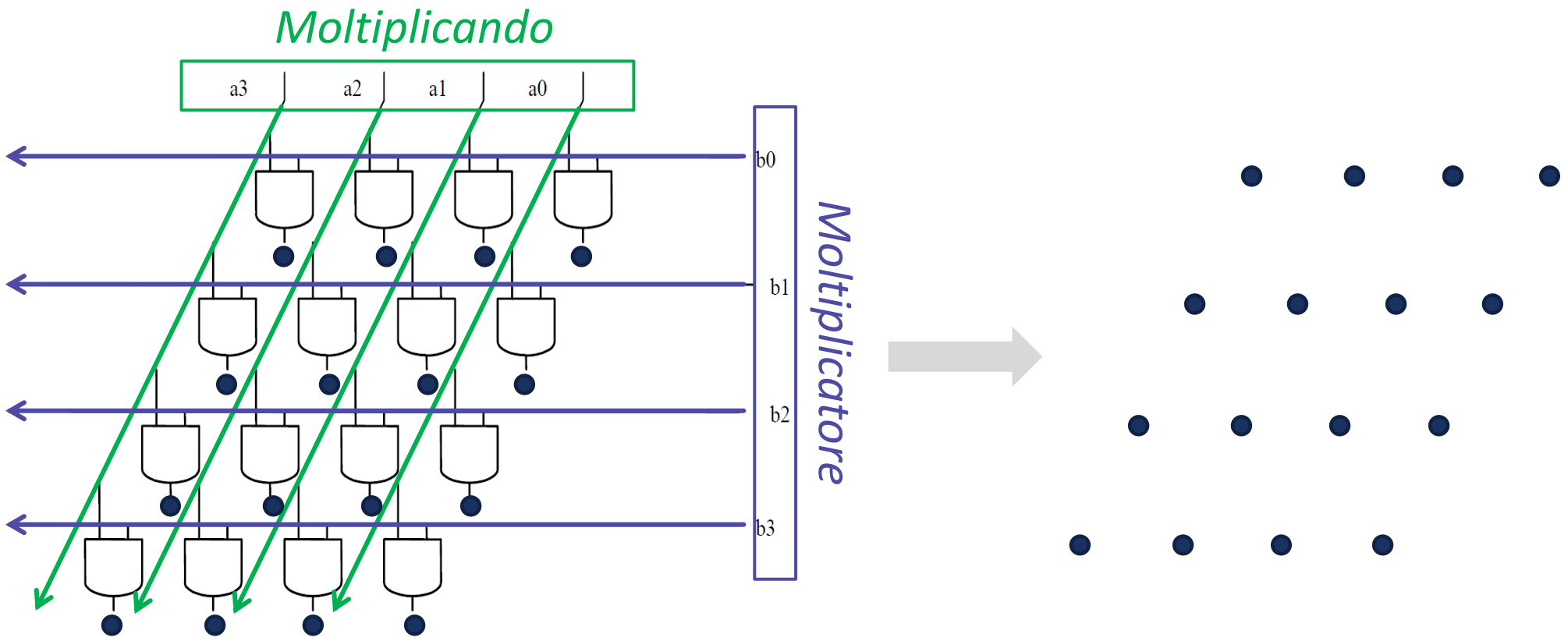
Prodotto come somma per colonne

E' la soluzione che più si avvicina alla procedura di moltiplicazione manuale tra numeri decimali interi → calcolata la matrice dei prodotti parziali, eseguiamo la somma di ciascuna colonna a partire da quella più a destra.



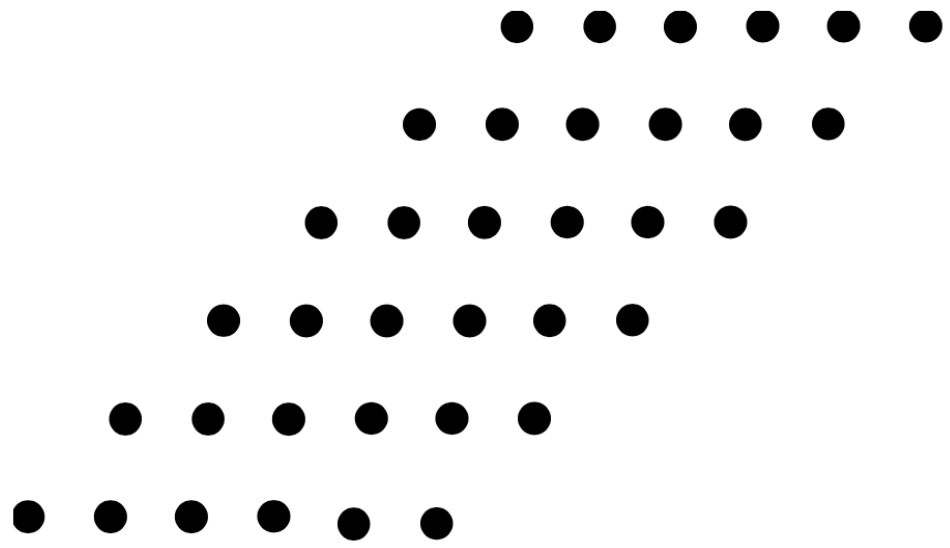
Prodotto come somma per colonne

Rappresentiamo ciascun prodotto parziale con un pallino, tenendo conto non del valore ma della **posizione** di ognuno di essi.



Prodotto come somma per colonne

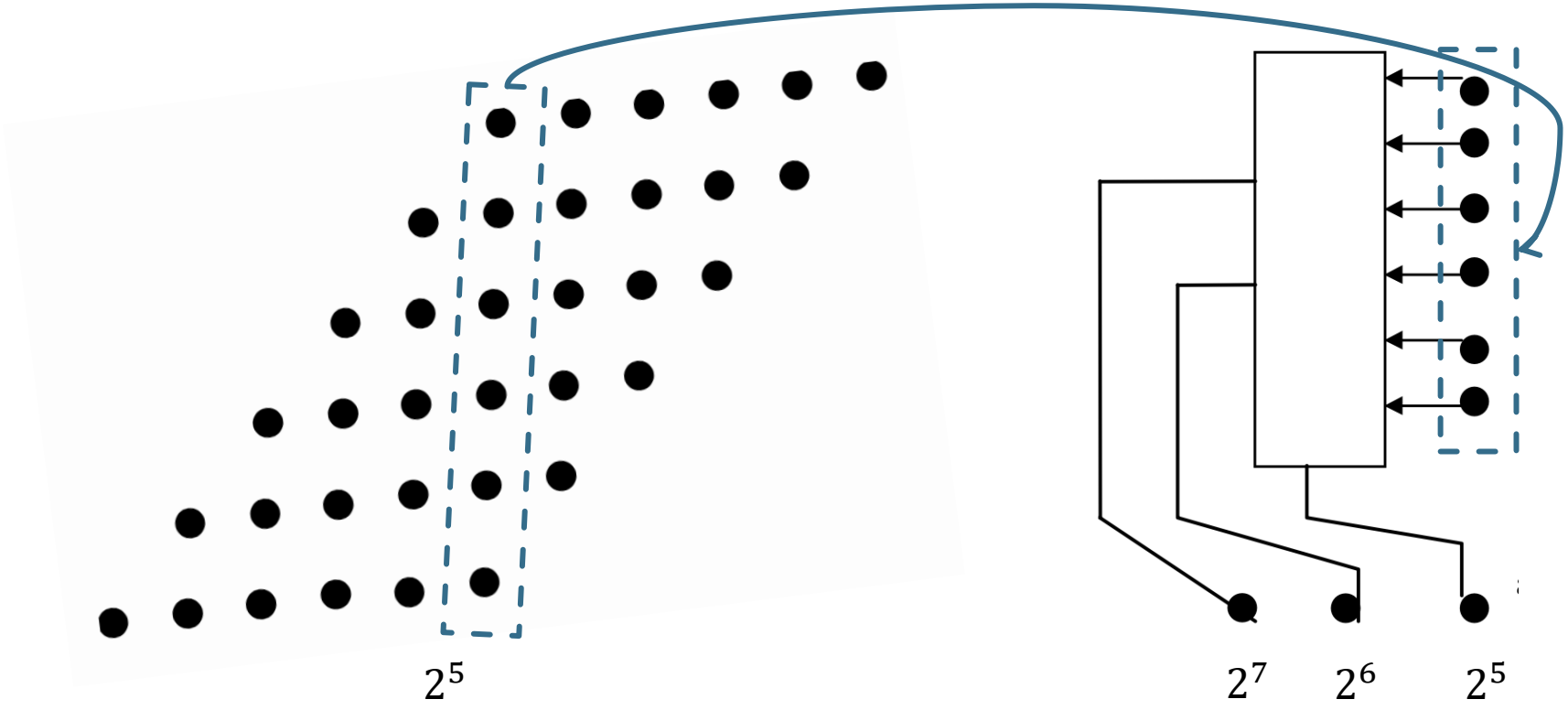
- Consideriamo un moltiplicando ed un moltiplicatore a 6 bit e utilizziamo la schematizzazione a pallini.
- Eseguire la somma per colonne significa sommare tutti i bit di ciascuna colonna → **contare** quanti di questi bit valgono 1.

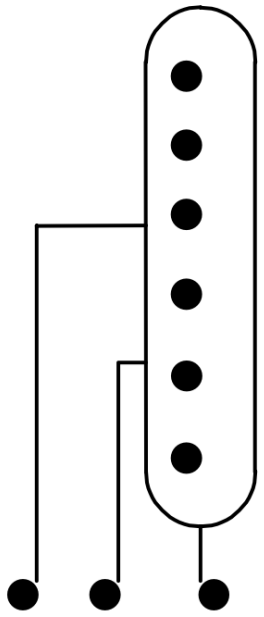


Il componente fondamentale per eseguire il conteggio è il circuito **contatore parallelo** che è un circuito *combinatorio* con **n** ingressi e $m = \lg(n+1)$ uscite.

Contatore parallelo

- Il contatore parallelo calcola la **codifica binaria** del numero di ingressi che, in un dato istante, valgono 1.





Modo alternativo di rappresentare il contatore parallelo.

Osservazioni

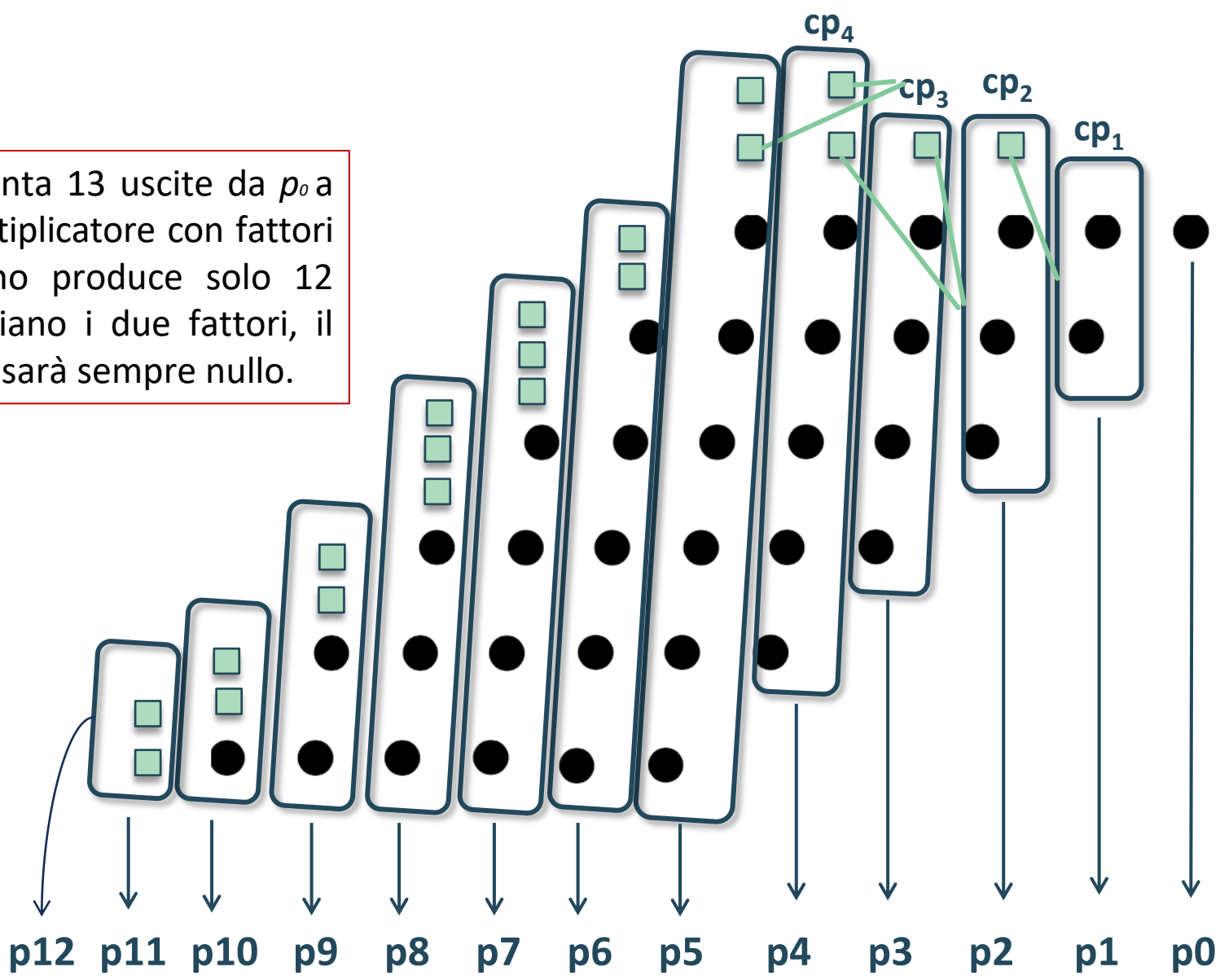
- Il legame tra il numero di entrate e il numero di uscite del contatore è:

n°entrate	2	3	4	5	6	7	8	9	..	14	15	16	..	30	31	32	..
n°uscite	2	2	3	3	3	3	4	4	..	4	4	5	..	5	5	6	..

- Non esiste un contatore ad un solo bit in ingresso.
- Un contatore a due ingressi ed uno a tre ingressi coincidono per definizione rispettivamente con un circuito **half-adder** e un circuito **full-adder**.

Gestire i riporti: primo circuito prodotto come somma per colonne

La struttura presenta 13 uscite da p_0 a p_{12} mentre un moltiplicatore con fattori di sei bit ciascuno produce solo 12 uscite; qualsiasi siano i due fattori, il tredicesimo bit p_{12} sarà sempre nullo.



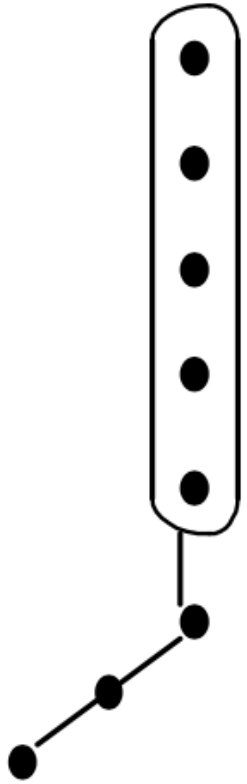
Osservazioni

- Perdita di regolarità dei circuiti e delle connessioni → contatori diversi per ogni colonna.
- Supponendo che un HA/FA o contatore parallelo con un qualsiasi numero di entrate presenti le uscite con un ritardo pari a un tempo elementare *dopo* il più lento degli ingressi, la somma dei bit della matrice iniziale richiede **2n-1** tempi elementari per la somma per colonne.

Gestire i riporti: secondo circuito prodotto come somma per colonne

- Per migliorare la velocità, consideriamo un secondo metodo che fa utilizzo della rappresentazione del contatore parallelo in figura.

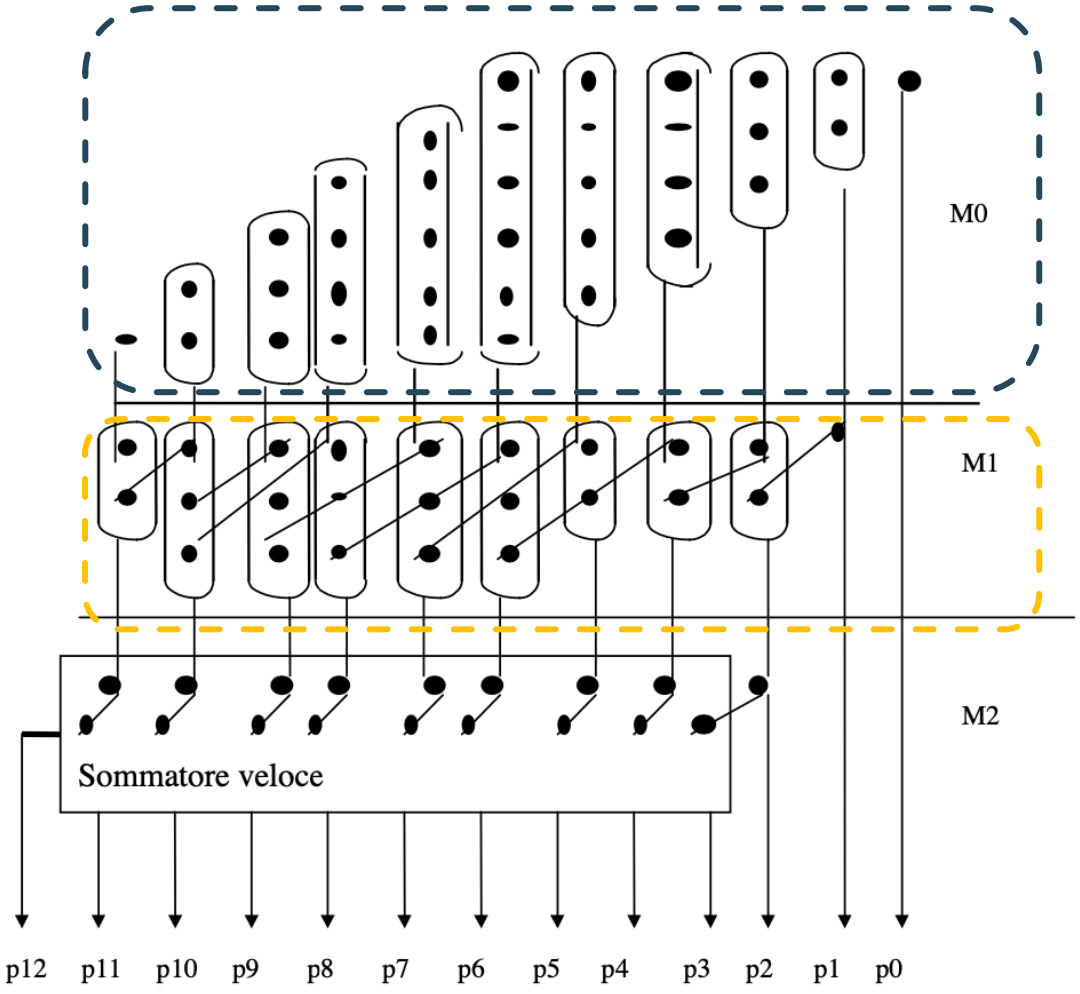
  - La linea chiusa rappresenta il contatore;
  - I cerchi pieni interni al simbolo del contatore sono le entrate (allineati sulla stessa verticale in quanto hanno lo stesso peso);
  - Le tre uscite (su colonne adiacenti in quanto hanno pesi diversi) sono unite da una linea per indicare che derivano dallo stesso contatore.



Secondo circuito prodotto come somma per colonne

- Avendo calcolato la matrice dei prodotti parziali (con la matrice di circuiti AND), il moltiplicatore realizza la somma per colonne come in figura.

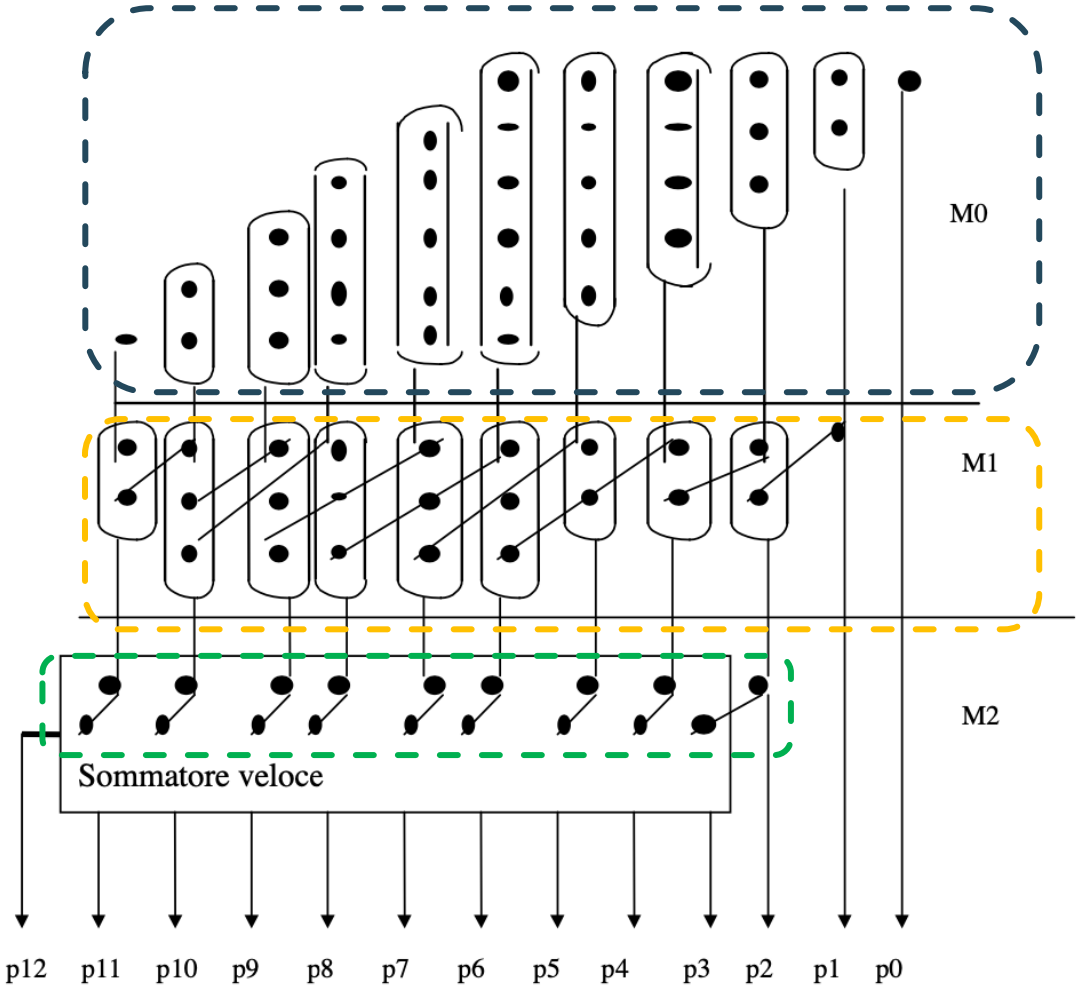
I *soli* bit della matrice iniziale vanno in ingresso a una serie di contatori (matrice **M0**) le cui uscite costituiscono gli ingressi a una seconda matrice di contatori **M1**.



Secondo circuito prodotto come somma per colonne

- Il procedimento viene ripetuto per passare dalla matrice M1 (tre righe) alla matrice M2 (due sole righe).

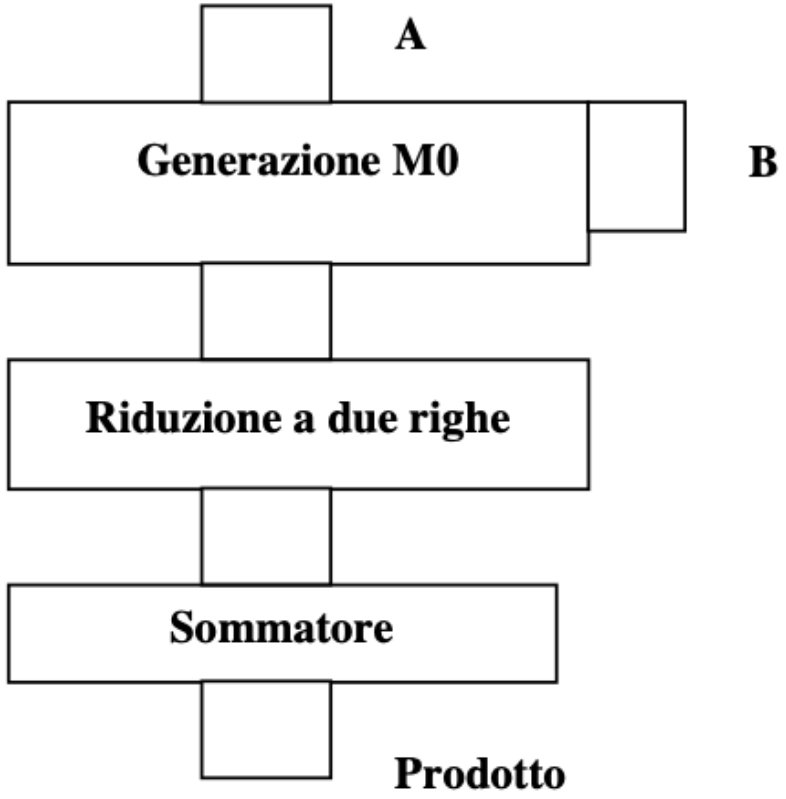
Raggiunta una matrice di due sole righe, la somma può essere eseguita mediante un normale addizionatore.



Osservazioni

- Il prodotto viene ottenuto in tre fasi successive:

1. La prima fase determina, mediante una matrice di circuiti AND, la matrice ***M0*** iniziale;
2. La seconda fase utilizza contatori parallelo per ridurre il numero di righe della matrice iniziale fino ad arrivare ad una matrice composta da due sole righe;
3. La terza fase utilizza un addizionatore per sommare le due righe dell'ultima matrice ottenuta alla seconda fase.



Tutto il ritardo è dovuto all'addizionatore finale: se si utilizza un addizionatore più complesso, ma molto più veloce di uno a propagazione di riporto, l'intero moltiplicatore ha tempo di esecuzione dell'ordine di **$\log_2(n)$**. Richiede **un solo addizionatore**.

MOLTIPLICATORE A CELLE MAC (MULTIPLY AND ACCUMULATE)

- Sfruttando due proprietà della matrice dei prodotti parziali, è possibile realizzare una **cella MAC** composta da una porta AND per il prodotto e un FA per la somma.
- È un'architettura più regolare del prodotto come somma per righe e diagonali (perché queste prevedono stadi diversi per fare le somme) ed iterativa.

Moltiplicatore a celle MAC (Multiply and Accumulate)

Ogni elemento della matrice dei prodotti parziali deve essere sommato con tutti gli elementi che stanno sopra nella stessa colonna e con tutti i riporti che eventualmente vengono da destra.

Moltiplicando

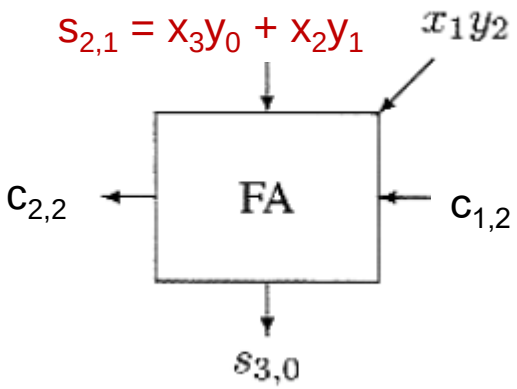
			x_3	x_2	x_1	x_0	
			x_3y_0	x_2y_0	x_1y_0	x_0y_0	y_0
		x_3y_1	x_2y_1	x_1y_1	x_0y_1		y_1
	x_3y_2	x_2y_2	x_1y_2	x_0y_2			y_2
x_3y_3	x_2y_3	x_1y_3	x_0y_3				y_3

Moltiplicatore

Il valore Y (moltiplicatore) è distribuito alla stessa maniera su tutte le righe, mentre il valore X cammina sulle diagonali.

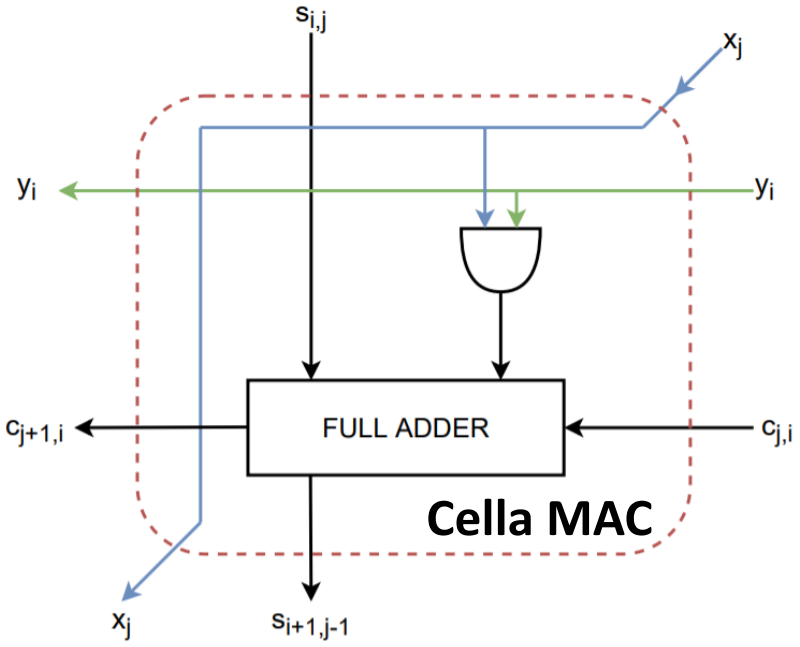
Osservazioni

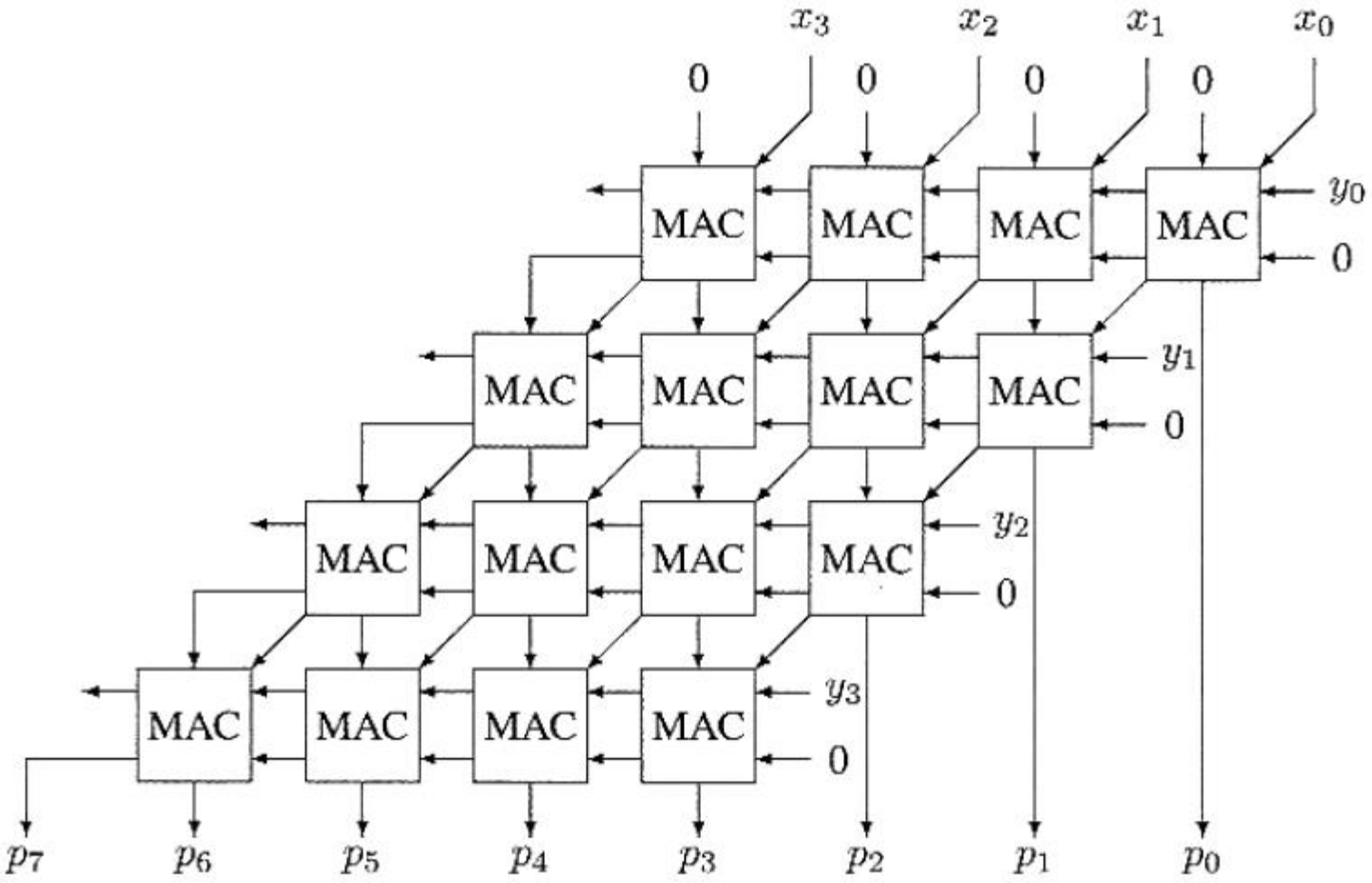
- Le operazioni che coinvolgono un singolo termine prodotto sono:
  1. Calcolo del prodotto stesso
  2. Somma parziale con gli altri termini che si trovano sulla stessa colonna
  3. Somma con il riporto entrante dallo stadio precedente.



- In uscita per quel termine occorre produrre:
  1. La nuova somma parziale
  2. Il riporto da propagare alla somma immediatamente a sinistra.

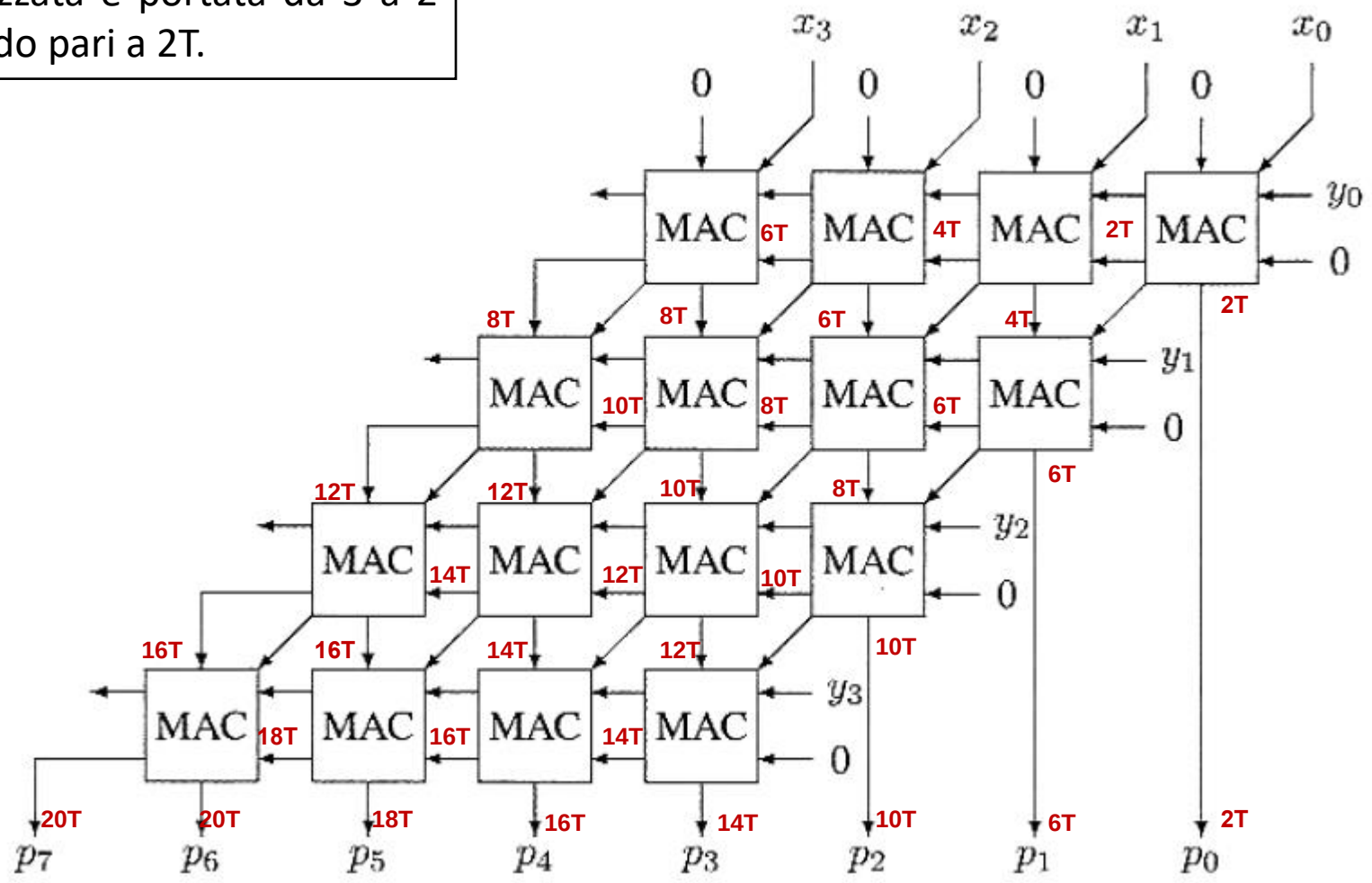
Moltiplicatore a celle MAC





Moltiplicatore a celle MAC: calcolo dei tempi

La rete corrispondente alla singola cella può essere ottimizzata e portata da 3 a 2 livelli, con un ritardo pari a 2T.



OPERAZIONI ARITMETICHE SU NUMERI INTERI

MOLTIPLICATORI SEQUENZIALI

Algoritmo manuale

$X=X_{n-1}\dots X_0 \quad Y=Y_{m-1}\dots Y_0 \quad X>0,\ Y>0$

L'operazione **X x Y** prevede il calcolo di una serie di prodotti $\mathbf{x_j\,Y}$ in cui il moltiplicando Y viene moltiplicato per il bit j-esimo del moltiplicatore X, per j=0..n-1.

Ciascun prodotto deve essere opportunamente shiftato di j posizioni a sinistra per poter concorrere al calcolo del prodotto finale attraverso una somma di tutti i prodotti j-esimi.

1010	Y (moltiplicando)
1101	X (moltiplicatore)
1010	x_0Y
0000	x_1Y*2
1010	x_2Y*2^2
1010	x_3Y*2^3
10000010	P

$$P=\sum_{j=0}^{n-1} x_j\, 2^j\, Y$$

Calcolo del prodotto parziale

1010	Y (moltiplicando)
1101	X (moltiplicatore)
00000000	$P_0=0$
1010	x_0Y
00001010	$P_1=P_0+x_0Y$
0000	x_1Y*2
00001010	$P_2=P_1+x_1Y*2$
1010	x_2Y*2^2
00110010	$P_3=P_2+x_2Y*2^2$
1010	x_3Y*2^3
10000010	$P_4=P_3+x_3Y*2^3=P$

Per evitare di dover memorizzare tutti i prodotti Pj per la somma finale, ad ogni passo si può calcolare una somma parziale dei prodotti data da:

$$P_{i+1} = P_i + x_i 2^i Y$$

$$P_0 = 0$$

Ad ogni passo è necessario effettuare un'operazione di shift di i posizioni

Algoritmo alternativo

Al passo (*i+1)-esimo* invece della sequenza di operazioni:

1. moltiplicazione di x_i per Y,
2. shift a sinistra di i posizioni del prodotto parziale,
3. somma del prodotto parziale shiftato con la somma parziale P_i per calcolare P_{i+1},

è possibile considerare una versione alternativa dell'algoritmo che considera la nuova sequenza di operazioni:

1. moltiplicazione di x_i per Y,
2. somma del prodotto parziale con P_i calcolato al passo precedente
3. shift a destra di una posizione della somma parziale calcolata al punto 2 per calcolare P_{i+1}

$$P_i := P_i + x_i Y;$$

$$P_{i+1} := 2^{-1} P_i$$

N.B: L'algoritmo alternativo è del tutto equivalente a quello derivato dalla procedura manuale, ma ha il vantaggio che ad ogni passo si effettua sempre uno shift di una sola posizione a destra

Algoritmo alternativo

Y 1010		
X 1101		
0000 0000		P0
1010		
0000 1010		ADD;*shift*
000 0101	0	P1
0000		
000 0101	0	ADD;*shift*
00 0010	10	P2
1010		
00 1100	10	ADD;*shift*
0 0110	010	P3
1010		
1 0000	010	ADD;*shift*
1000	0010	
A	Q	

Qualche considerazione:

- Da un punto di vista architetturale mi servirebbero: dei registri per contenere X e Y, delle porte AND per ottenere il prodotto x_iY, un sommatore di stringhe per calcolare la somma parziale, e uno shift register per memorizzare questa somma e shiftarla a destra. Si consideri che:
  - Il valore da sommare alla somma parziale è Y oppure 0 in base a X[0] → è possibile usare un MUX invece delle porte AND
  - Il sommatore ogni volta lavora solo con n/2 bit della somma parziale che è da n bit → mi serve un sommatore con parallelismo n/2
  - Ad ogni iterazione uso solo X[0], che poi non mi serve più → il registro che contiene X può essere sfruttato per contenere altri dati man mano che si procede nell'algoritmo→ uso 2 registri concatenati A e Q: A rappresenta la porzione di somma parziale che utilizzo ogni volta nella somma, mentre Q all'inizio contiene X ma poi pian piano contiene i bit della somma parziale che vengono «buttati fuori» dagli shift. Alla fine il risultato lo leggo su A.Q

Prodotto di numeri interi relativi

- Per il calcolo del prodotto di due interi relativi **non è possibile applicare l'algoritmo** derivato dalla procedura manuale, che fornisce un risultato errato

- Se i numeri sono codificati in modulo e segno l'unica possibilità è quella di calcolare separatamente il prodotto dei moduli e dei segni

- **Se i numeri sono codificati in complementi a due** una soluzione concettualmente semplice consiste nel negare tutti gli operandi negativi, effettuare un'operazione unsigned sui numeri positivi risultanti e poi negare il risultato se necessario

Problema: **sono necessari ulteriori cicli di clock per negare X, Y e il risultato a doppia lunghezza P**

Numeri relativi in complementi a due: considerazioni

Una soluzione alternativa a quella vista si basa su alcune proprietà della rappresentazione in complementi:

$$-X = x'_{n-1}\, x'_{n-2} \ldots x'_1 x'_0 + 000\ldots1 \pmod{2^n}$$
$$\text{ma } x'_i = 1 - x_i \pmod 2$$

$$-X = (111\ldots11 - x_{n-1}\, x_{n-2} \ldots x_1 x_0) + 000\ldots01 \pmod{2^n}$$

Se X>0 possiamo scrivere $X = \sum_{i=0}^{n-2} 2^i x_i$ poiché $x_{n-1} = 0$

Se X<0 la relazione trovata non è valida, mentre visto che $x_n = 1$ si può scrivere:

$$-X = 111\ldots11 - (0\, x_{n-2} \ldots x_1 x_0 + 100\ldots00) + 000\ldots01 =$$
$$= (111\ldots11 - 100\ldots00 + 000\ldots01) - x_{n-2} \ldots x_1 x_0 =$$
$$= 100\ldots00 - x_{n-2} \ldots x_1 x_0 =$$
$$= 2^{n-1} - x_{n-2} \ldots x_1 x_0 \Rightarrow$$

Se X<0 → $X = -2^{n-1} + x_{n-2} \ldots x_1 x_0 = -2^{n-1} + \sum_{i=0}^{n-2} 2^i x_i$

Moltiplicatori in complementi a due

Se X>0 $$X = \sum_{i=0}^{n-2} 2^i x_i$$

Se X<0 $$X = -2^{n-1} + \sum_{i=0}^{n-2} 2^i x_i$$

$$\boxed{X = -2^{n-1} x^{n-1} + \sum_{i=0}^{n-2} 2^i x_i}$$

L'equazione trovata implica che **possiamo trattare i bit $x_{n-2}x_{n-3}...x_1x_0$ di un intero in complemento a due come se appartenessero ad un numero unsigned**. Inoltre:

- Per un numero positivo viene assegnato peso 2^{n-1} al bit di segno x_{n-1}: poiché è nullo il suo contributo al numero è 0 sulla cifra più significativa
- Per un numero negativo viene assegnato peso -2^{n-1} al bit di segno x_{n-1}: poiché esso vale 1 il suo contributo al numero è -1 sulla cifra più significativa

Con questo schema è possibile utilizzare una tecnica di moltiplicazione unsigned con un'unica modifica: ***quando si moltiplica il bit di segno è necessario effettuare una sottrazione*** invece di un'addizione nel passo finale se si incontra un segno negativo.

Modifiche all'algoritmo manuale

A seconda dei segni dei due fattori X(moltiplicatore) e Y(moltiplicando) si possono avere 4 casi:

- **X>0, Y>0** : moltiplicazione fra unsigned, effettuata con passi add e shift
- **X>0, Y<0** : ogni volta che si moltiplica Y per $x_j \neq 0$ il prodotto parziale è negativo, quindi il bit più significativo di A deve essere sempre 1: è possibile usare un latch in cui viene inserito il valore 1 e che viene usato per «alimentare» lo shift register in tutte le iterazioni
- **X<0, Y>0** : per i primi n-1 prodotti il prodotto parziale è positivo, mentre solo per l'n-esimo prodotto è necessario effettuare un passo di correzione con la sottrazione A-M
- **X<0, Y<0** : la procedura segue il caso 2: il bit più significativo di A è 0 finché $x_j=0$ e diventa 1 quando moltiplico $x_j=1$ per Y; anche in questo caso per l'n-esimo prodotto è necessario effettuare un passo di correzione con la sottrazione A-M

- Il caso 2 viene gestito grazie a un latch F che viene messo a 1 quando il moltiplicando è negativo (M[7]=1) e la cifra corrente del moltiplicatore (Q[0]) è 1
- Il caso 3 viene gestito effettuando la correzione finale

Esempio: X<0, Y>0

X=-5=1011 (not (0101) + 0001)
Y=3=0011

P4=R=-15=11110001= (not(00001111)+00000001)

Y 0011		
X 1011		
0000 0000		P0
0011		
0000 0011		ADD;*shift*
000 0001	1	P1
0011		
000 0100	1	ADD;*shift*
00 0010	01	P2
0000		
00 0010	01	ADD;*shift*
0 0001	001	P3
0011		
1 1110	001	**SUB**;*shift*
1111	0001	
A	Q	

Esempio: X>0, Y<0

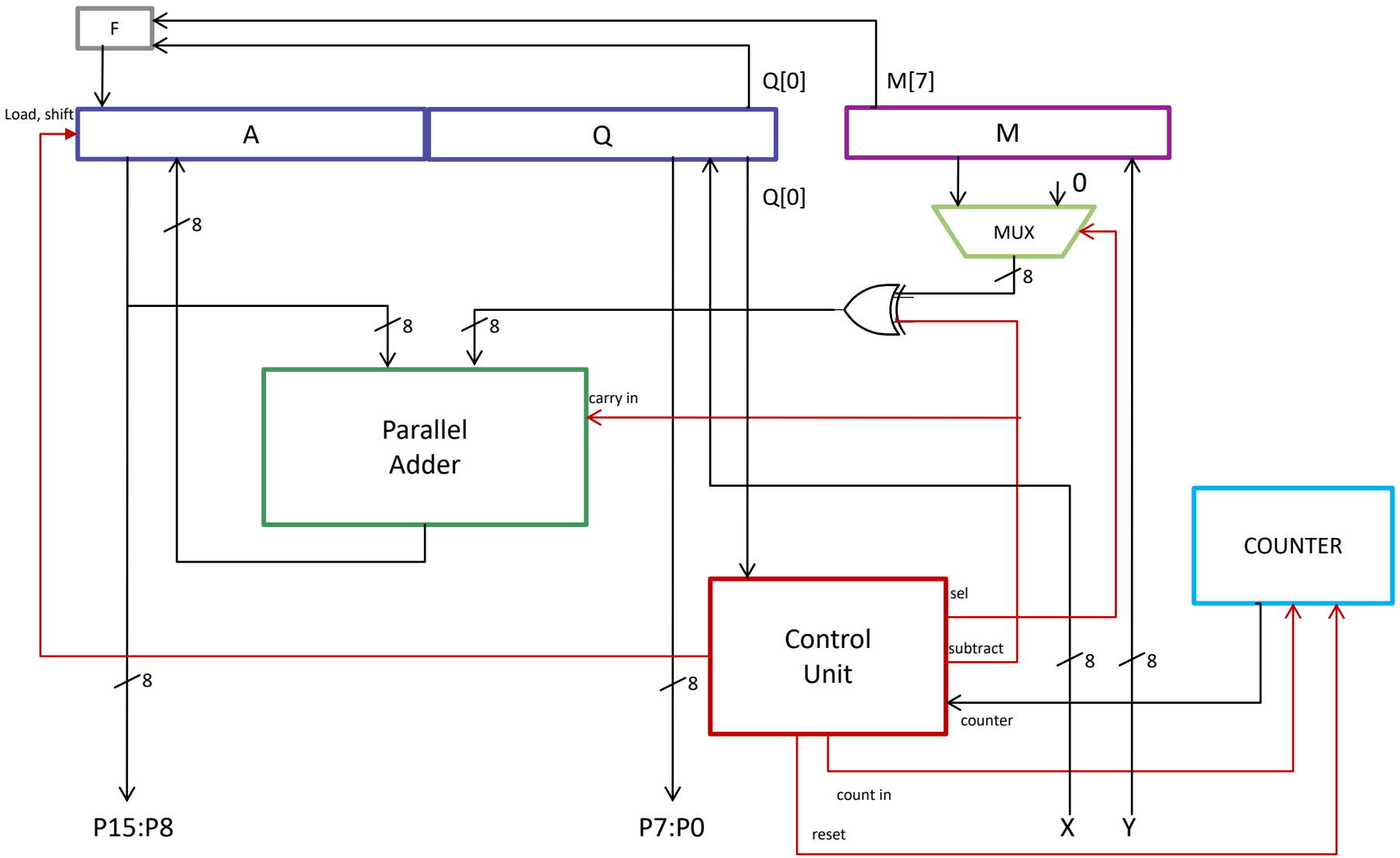
Y 1101		
X 0101		
0000		P0
1101		
1101		ADD;*shift*
1110	1	P1
0000		
1110	1	ADD;*shift*
1111	01	P2
1101		
1100	01	ADD;*shift*
1110	001	P3
1101		
1110	001	**SUB**;*shift*
1111	0001	
A	Q	

X=5=0101
Y=-3=1101= (not (0011) + 0001)

P4=R=-15=11110001= (not(00001111)+00000001)

In questo passo, siccome x0=1 ci rendiamo conto che il moltiplicando è negativo (M[3]=1):
Settiamo il flip flop F a 1, e da questo momento in poi nel fare il right shift inseriamo in testa al registro A un 1

Moltiplicatore di Robertson in complementi a due



Algoritmo di Robertson per operandi interi relativi ad 8 bit

```
2CMultiplier:     (in:INBUS; OUT:OUTBUS)
                  register A[7:0],M[7:0],Q[7:0],COUNT[2:0],F;
                  bus INBUS[7:0],OUTBUS[7:0];

BEGIN:            A:=0,COUNT:=0,F:=0,
INPUT:            M:=INBUS;Q:=INBUS;

ADD:              A[7:0]:= A[7:0] + M [7:0] x Q[0],
                  F:= (M[7] and Q[0]) or F;
RSHIFT:           A[7]:= F,   A[6:0].Q:= A.Q[7:1],
INCREMENT:        COUNT:=COUNT+1

TEST:             if COUNT<7 then go to ADD;

SUBTRACT:         A[7:0]:=A[7:0]-M[7:0]xQ[0];      {l'ultima op è sempre SUB}
RSHIFT:           A[7]:= A[7],   A[6:0].Q:= A.Q[7:1];

OUTPUT:  OUTBUS;=Q; OUTBUS:=A;
END 2CMultiplier;
```

Codifica di Booth

Si consideri un intero X la cui rappresentazione in complementi a 2 sia $x_{n-1}x_{n-2}....x_0$ e si definisca:

$$y_0 = -x_0$$
$$y_1 = -x_1 + x_0$$
$$y_2 = -x_2 + x_1$$
$$....$$
$$y_{n-1} = -x_{n-1} + x_{n-2}$$

Moltiplicando la prima equazione per 2^0, la seconda per 2^1, la terza per 2^2 e così via, e sommando le n equazioni, si ottiene:

$$\boxed{y_{n-1}2^{n-1} + y_{n-2}2^{n-2} + \cdots + y_0 2^0} = \boxed{-x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + .. + x_0 2^0}$$

Rappresentazione posizionale

Rappresentazione in complementi a due

Codifica di Booth

Il vettore $y_{n-1}y_{n-2}....y_0$, i cui elementi appartengono a {-1,0,1}, è la rappresentazione Booth-1 di x e:

$$x = y_{n-1}2^{n-1} + y_{n-2}2^{n-2} + \cdots + y_0 2^0$$

x può essere rappresentato in maniera alternativa utilizzando la sua rappresentazione Booth

La rappresentazione Booth-1 di x può essere facilmente ottenuta sostituendo ciascuna coppia di bit adiacenti di x con un valore in {-1,0,1}, secondo la corrispondenza in tabella (si considera uno zero in coda alla stringa x per valutare la coppia $x_0\, x_{-1}$)

$x_j\, x_{j-1}$	codifica
00/11	0
01	+1
10	-1

1 0 1 1 0 0 1 1 0

-codifico:

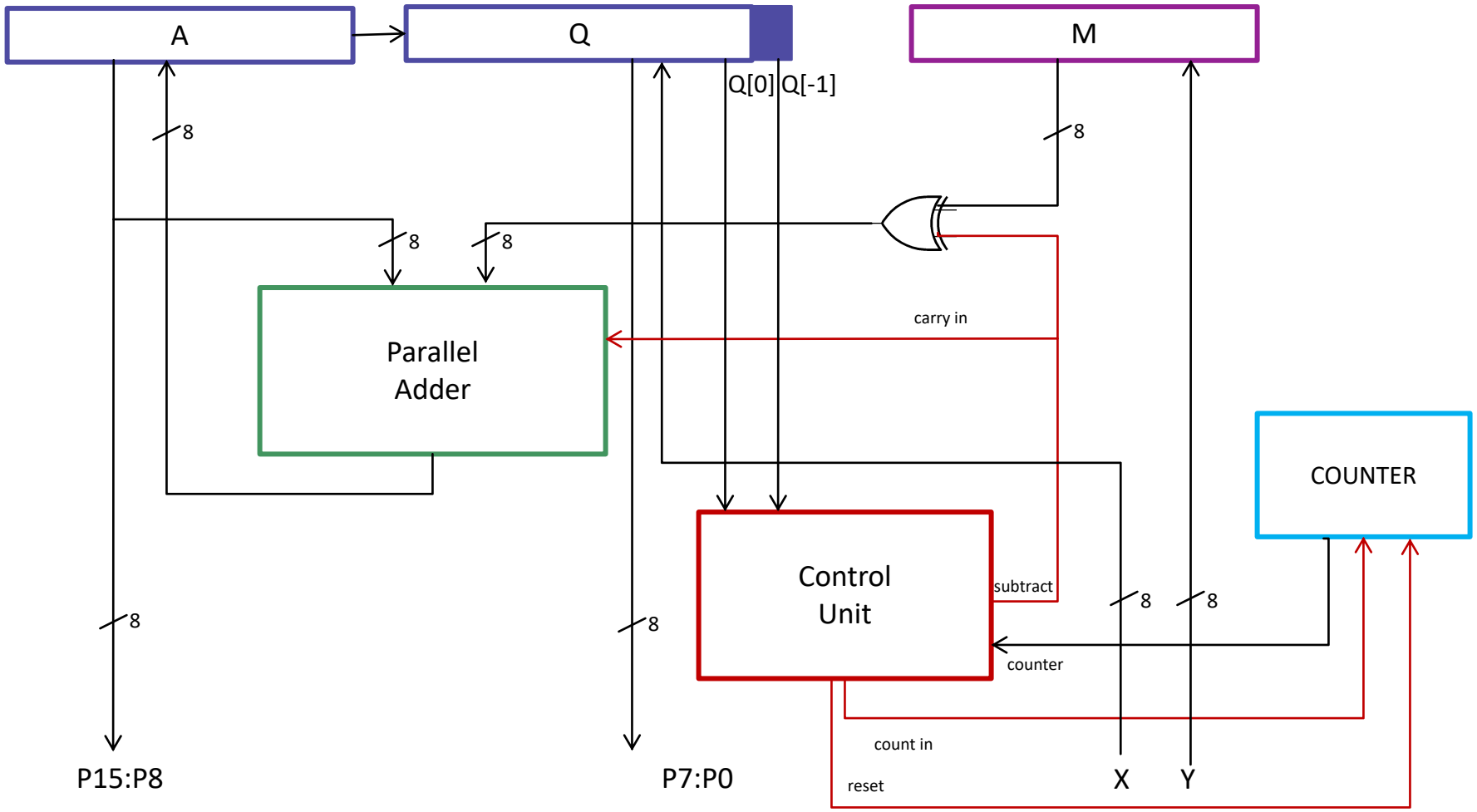
-1 +1 0 -1 0 +1 0 -1

Moltiplicatore di Booth

- Sfruttando la codifica di Booth è possibile ridurre il numero di moltiplicazioni (e di somme) da effettuare per il prodotto fra numeri relativi.
- Facendo riferimento alla struttura sequenziale vista per il moltiplicatore di Robertson, poiché ogni cifra della rappresentazione Booth-1 del moltiplicatore appartiene a {-1,0,1}, **a seconda della cifra coinvolta nel prodotto verrà effettuata un'operazione di somma o di differenza (<u>o nessuna delle due</u>) prima dello shift.**
- Mentre nell'algortimo di Robertson viene esaminato il moltiplicatore $x = x_{n-1}..x_j x_{j-1}..x_0$ da destra verso sinistra e si considera il bit j-esimo per determinare quale azione effettuare, nel'algoritmo di Booth ad ogni passo si esaminano 2 bit adiacenti, $x_j x_{j-1}$:

x_j x_{j-1}	
00/11	Non viene effettuata ne la somma ne la sottrazione, ma solo lo shift
01	Y viene aggiunto al prodotto parziale corrente
10	Y viene sottratto dal prodotto parziale corrente

Moltiplicatore di Booth



Algoritmo di Booth per operandi interi di 8 bit

```
BoothMultiplier:   (in:INBUS; OUT:OUTBUS)
                   register A[7:0],M[7:0],Q[7:-1],COUNT[2:0];
                   bus INBUS[7:0],OUTBUS[7:0];

BEGIN:             A:=0,COUNT:=0;
INPUT:             M:=INBUS; Q[7:0]:=INBUS;Q[-1]:=0;

SCAN:              if Q[0]Q[-1] = 01
                       then A[7:0]:= A[7:0] + M [7:0];
                   else if Q[0]Q[-1] = 10
                       then A[7:0]:= A[7:0] - M[7:0];

RSHIFT:            A[7]:= A[7],   A[6:0].Q:= A.Q[7:0];
INCREMENT:         COUNT:=COUNT+1; go to SCAN;

TEST:              if COUNT<8 then go to SCAN;

OUTPUT: OUTBUS;=A;
                   OUTBUS:=Q[7:0];
END BoothMultiplier;
```

OPERAZIONI ARITMETICHE SU NUMERI INTERI

DIVISORI

Divisioni fra numeri interi: considerazioni generali

D dividendo $D_{m-1}....D_0$ (m bit)
$V \neq 0$ divisore $V_{n-1}.....V_0$ (n bit)

Si vuole determinare un quoziente Q e un eventuale resto R in modo che sia

$$D = Q \times V + R \quad \text{con } 0 \leq R < V$$

- Se V è espresso su n bit, allora poiché il resto può assumere al massimo il valore R=V-1, **R è anche esso espresso su al più <u>n bit</u>**

- Il massimo quoziente si ha col minimo divisore, ossia con V=1: in questo caso Q coincide con D e quindi se D è espresso su m bit anche Q sarà espresso su al più m bit
  - in generale **Q è espresso su al più <u>m-n+1</u> bit**

NOTA: stiamo considerando i bit veri del numero, cioè assumiamo che il primo degli n/m bit sia non nullo

Divisioni fra numeri interi: algoritmo manuale

```
dividendo        divisore
 1 1 1 0 1 1      1 0 1
 1 0 1            1 0 1 1
 ------------     quoziente
   1 0 0
   0 0 0
 ------------
   1 0 0 1
     1 0 1
 ------------
     1 0 0 1
       1 0 1
 ------------
       1 0 0
       resto
```

Nel procedimento manuale di divisione il dividendo viene scandito da sinistra verso destra, e i bit del quoziente vengono determinati uno alla volta a partire da quello più significativo, procedendo con una serie di confronti e sottrazioni.

Passo iniziale:

1) Si <u>confrontano</u> gli n bit più significativi del dividendo (dividendo parziale D0) con gli n bit del divisore;
2) Si calcola la prima cifra (da sinistra) del quoziente q_0, che sarà 1 oppure 0 a seconda che il dividendo parziale D0 contenga o no il divisore;
3) Si effettua la <u>sottrazione</u> fra il dividendo parziale D0 e il prodotto q_iV, determinando il primo resto parziale R_1

Generico passo i:

1) Si pone $D_i = R_i$,con R_i resto parziale determinato al passo i-1; si <u>confronta</u> Di con il divisore;
2) Si calcola la i-esima cifra (da sinistra) del quoziente qi, che sarà 1 oppure 0 a seconda che il dividendo parziale Di contenga o no il divisore;
3) Si effettua la <u>sottrazione</u> fra il dividendo parziale Di e il prodotto q_iV <u>shiftato</u> a destra di i posizioni, determinando il nuovo resto parziale R_{i+1}

La procedura termina quando il dividendo è stato scandito completamente: il resto parziale Ri determinato in questo passo costituisce il resto finale R della divisione

Divisioni fra numeri interi: passo i-esimo dell'algoritmo manuale

D dividendo $D_{m-1}....D_0$ (m bit)
V≠0 divisore $V_{n-1}.....V_0$ (n bit)

si noti che all'inizio dell'algoritmo manuale è necessario allineare il divisore al dividendo parziale shiftandolo a sinistra di **m-n** posizioni

Al passo i-esimo (con i=0…m-n) vengono effettuate le seguenti operazioni:

1) Si confronta Ri=Di con V: se Di contiene V, la cifra i-esima (da sinistra) del quoziente qi sarà 1, altrimenti sarà 0

2) Si calcola il nuovo dividendo parziale: $$R_{i+1} = R_i - q_i 2^{-i} V$$

Ad ogni passo è necessario effettuare un'operazione di shift del divisore di i posizioni a destra e una sottrazione

Divisione: algoritmo manuale

Consideriamo V e Q espressi su **n** bit e D su (**2n-1**) bit

D	**1 1 1 1 0 1 1**	**1 0 1 0** **V**
$R0=D_7D_6D_5D_4$	1 1 1 1	1 1 0 0
conf	$\mathbf{q_3=1}$	Q
$V^*=2^3*V$	1 0 1 0	
$R1=R0-V^*q_3$	0 1 0 1 0 1 1	
conf	$\mathbf{q_2=1}$	
$V^*=2^2*V$	1 0 1 0	
$R2=R1-V^*q_2$	0 0 0 1 1	
conf	$\mathbf{q_1=0}$	
$V^*=2*V$	1 0 1 0	
$R3=R2-V^*q_1$	0 0 1 1	
conf	$\mathbf{q_0=0}$	
$V^*=2*V$	1 0 1 0	
$R4=R3-V^*q_0$	**0 0 1 1**	
	R	

$$R_{i+1} = R_i - q_i 2^{-i} V$$

123/10=> Q=12, R=3

Divisione: algoritmo alternativo

Al passo *i-esimo* invece della sequenza di operazioni:

1. confronto di R_i con V per determinare q_i
2. prodotto di V per q_i e shift di i posizioni a destra
3. sottrazione del prodotto shiftato da R_i per calcolare R_{i+1}

è possibile considerare una versione alternativa dell'algoritmo che considera la nuova sequenza di operazioni:

1. left shift di una posizione di R_i
2. confronto di $2R_i$ con V per determinare q_i
3. sottrazione del prodotto q_iV da $2R_i$

$$R_i := 2R_i; R_{i+1} := R_i - q_iV;$$

$$R_{i+1} := 2R_i - q_iV;$$

NB: l'algoritmo parte con $R_0=2R_0$

L'algoritmo alternativo è del tutto equivalente a quello derivato dalla procedura manuale, ma ha il vantaggio che ad ogni passo si effettua sempre uno shift di una sola posizione a sinistra

Divisione: algoritmo alternativo esempio 1

Consideriamo V e Q espressi su **n** bit e D su (**2n-1**) bit

D=R0	**1 1 1 1 0 1 1**	**1 0 1 0**	**V**
2R0	1 1 1 1	0 1 1 -	1 1 0 0
conf	1 0 1 0		
2R0-V*=R1	0 1 0 1	0 1 1 -	
2R1	1 0 1 0	1 1 - -	
conf	1 0 1 0		
2R1-V*=R2	0 0 0 0	1 1 - -	
2R2	0 0 0 1	1 - - -	
conf	0 0 0 0		
2R1-V*=R3	0 0 0 1	1 - - -	
2R3	0 0 1 1	- - - -	
conf	0 0 0 0		
2R1-V*=R4=R	0 0 1 1	- - - -	

Q

A=R Q

CONSIDERAZIONI

- ad ogni passo il dividendo parziale R_i viene shiftato a sinistra, e i suoi primi n bit vengono «confrontati» con V; se risultano maggiori, la cifra corrente di q viene posta a 1 e viene effettuata la sottrazione R_i-V
- Il dividendo può essere caricato in una coppia di registri **A.Q**: Q viene "svuotato" man mano che avvengono gli shift e può essere usato per memorizzare le cifre del quoziente calcolate a ogni passo. A conterrà il resto alla fine del processo.
- l'utilizzo della coppia A.Q come descritto in precedenza implica che Q[0] sia "vuoto" al momento della valutazione; già dal primo passo quindi, il dividendo deve essere shiftato a sinistra di una posizione: ciò implica che abbiamo a disposizione 1 bit in meno per codificare il dividendo

123/10=> Q=12, R=3

Divisione: algoritmo alternativo esempio 2

Consideriamo V e Q espressi su **n** bit e D su (**2n-1**) bit

D=R0		**1 0 0 0 0 1 1**	**1 0 0 1**	**V**
2R0		1 0 0 0	0 1 1 -	0 1 1 1
conf		0 0 0 0		
2R0-V*=R1		1 0 0 0	0 1 1 -	
2R1	1	0 0 0 0	1 1 - -	
conf		1 0 0 1		
2R1-V*=R2		0 1 1 1	1 1 - -	
2R2		1 1 1 1	1 - - -	
conf		1 0 0 1		
2R1-V*=R3		0 1 1 0	1 - - -	
2R3		1 1 0 1	- - - -	
conf		1 0 0 1		
2R1-V*=R4=R		0 1 0 0	- - - -	
		A=R	Q	

Nota: l'algoritmo mostrato nell'esempio riprende la procedura manuale in cui idealmente si fa uso di comparatori e si abbassano le cifre necessarie ad avere una differenza non negative

67/9 => Q=7,R=4

Divisioni e comparatori

Come visto per la procedura manuale, l'operazione di divisione richiede una successione di sottrazioni e di confronti.
Idealmente il confronto potrebbe essere effettuato da un circuito **comparatore**, che riceve in ingresso due numeri interi A e B e produce due uscite α e β tali che:

$$\alpha = 1 \Rightarrow A>B$$
$$\beta = 1 \Rightarrow A<B$$
$$\alpha = \beta = 0 \Rightarrow A=B$$

Per realizzare il comparatore su n bit è possibile collegare opportunamente dei comparatori elementari che confrontano cifre di un solo bit.

Se ambedue i numeri A e B sono <u>interi senza segno</u> si verifica facilmente che <u>confrontando i bit di A e quelli di B a partire da sinistra verso destra</u> (iniziando quindi dai bit più significativi) <u>il risultato del confronto viene determinato non appena uno dei due numeri ha in posizione i-esima un bit di valore 0 e l'altro lo ha di valore 1</u>: *il numero col bit a 1 è certamente il maggiore, indipendentemente dai valori dei bit meno significativi.*

Divisione: confronto senza comparatore

Il problema centrale della divisione è il calcolo della cifra del quoziente q_i come confronto fra V e $2R_i$ al passo i-esimo:

- Se $V>2R_i \Rightarrow q_i=0$
- Se $V\leq 2R_i \Rightarrow q_i=1$

se V è costituito da un numero elevato di cifre l'utilizzo di comparatori potrebbe risultare oneroso in termini di circuiti logici.

q_i può essere calcolato sottraendo V da $2R_i$ ed esaminando il segno della differenza: *se è negativo $q_i=0$, altrimenti $q_i=1$*
Si noti che la differenza **$2R_i$ –V** andrebbe comunque calcolata se $q_i=1$ e in tal caso fornirebbe R_{i+1}

- I processi di determinazione di q_i e R_{i+1} possono essere fra loro combinati secondo due principali algoritmi: *restoring* and *non restoring*

Calcolo combinato di q_i e R_{i+1}

L'algoritmo di divisione prevede ad ogni passo i il calcolo di:

$$R_{i+1} := 2R_i - q_i V;$$

Come si è visto, q_i può essere determinato con la differenza

$$\Delta = 2R_i - V$$

Se $\Delta \geq 0$ allora q_i=1, e quindi effettivamente la differenza calcolata fornisce il valore di R_{i+1};

Se $\Delta < 0$ allora q_i=0 e quindi ho calcolato la quantità $2R_i$-V, mentre avrei dovuto calcolare $R_{i+1} = 2R_i$
Per avere il risultato corretto **è necessario in tal caso effettuare un'operazione di *restoring* che consiste nel sommare V alla quantità Δ calcolata:**

$$\boxed{\begin{aligned} \Delta \ := R_{i+1} &:= 2R_i - V \\ R_{i+1} &:= R_{i+1} + V \end{aligned}}$$

Restoring Division

La tecnica di **restoring division** esegue <u>sempre</u>, al passo i-esimo dell'algoritmo di divisione, la <u>sottrazione</u>

$$R_{i+1} = 2R_i - V$$

- se il risultato di tale differenza è <u>negativo,</u> allora $\mathbf{q_i=0}$ e si effettua un'operazione di "restoring" del valore di R_{i+1} sommando a esso il valore V <u>nello stesso passo</u>

$$R_{i+1} := 2R_i - V$$
$$R_{i+1} := R_{i+1} + V$$

- se il risultato della differenza è <u>positivo</u>, allora $\mathbf{q_i=1}$ e non ci sono ulteriori operazioni da eseguire nello stesso passo.

$$R_{i+1} := 2R_i - V$$

Non-Restoring Division

La tecnica di **non-restoring division** prende spunto dal fatto che a un'eventuale operazione di restoring (a) effettuata al passo i-esimo nel caso in cui q_i=0, segue sempre, al passo (i+1)-esimo, la sottrazione (b).

a) $R_i := R_i + V$

b) $R_{i+1} := 2R_i - V$

Le due operazioni possono essere fuse:

i): $\Delta_i = R_{i+1} = 2R_i - V < 0 \Rightarrow q_i = 0$ effettuo il restoring:

$$R_{i+1} = R_{i+1} + V$$

i+1): $\Delta_{i+1} = R_{i+2} = 2R_{i+1} - V = 2(R_{i+1} + V) - V = 2R_{i+1} + 2V - V =$

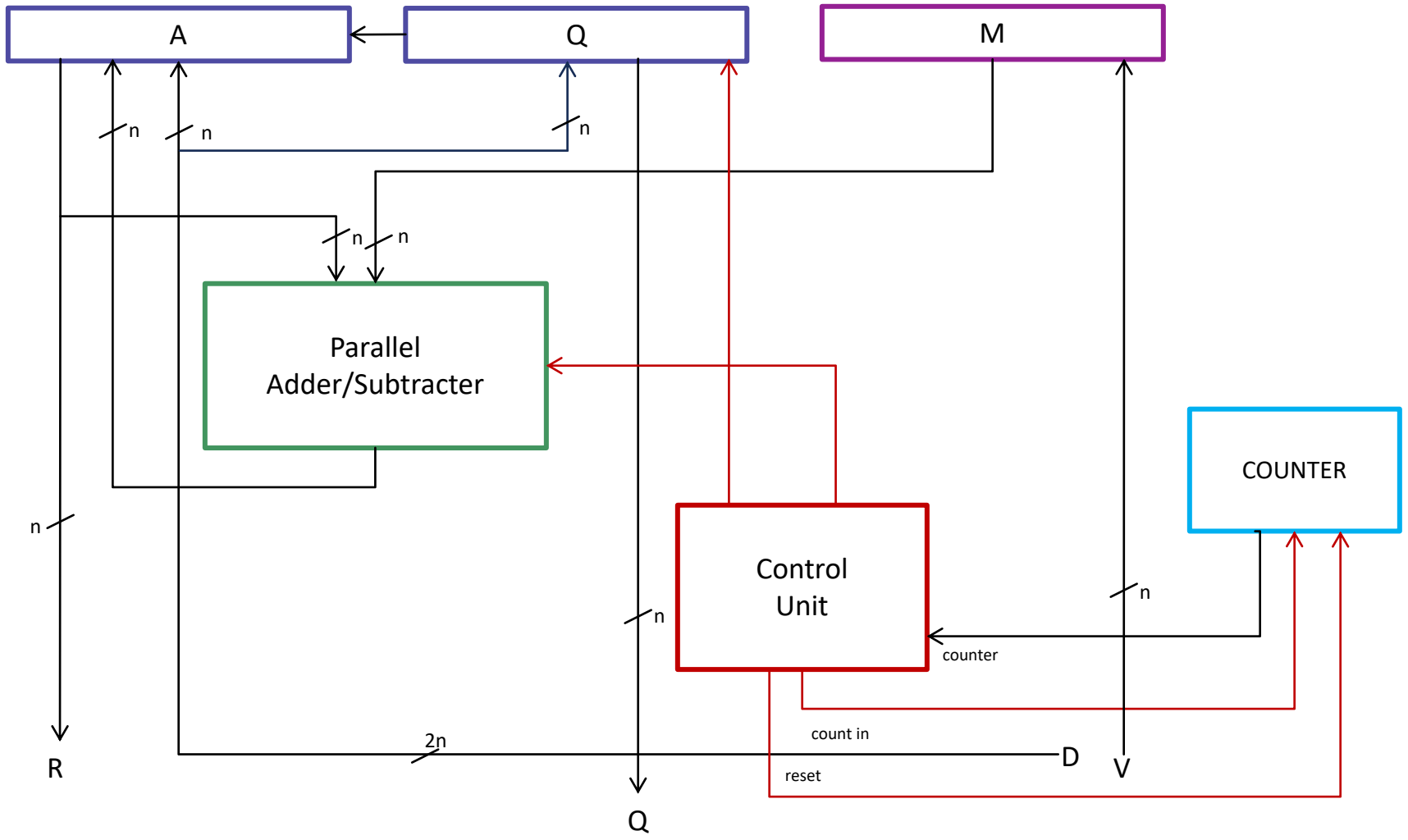
$$= \mathbf{2R_{i+1} + V}$$

Se al passo i, dopo aver calcolato la differenza ($2R_i$-V) risulta q_i=0, la prossima operazione eseguita sarà una somma:

$\Delta = \mathbf{2R_{i+1} + V}$

NOTA: L'algoritmo deve prevedere esplicitamente la possibilità di avere risultati negativi per la sottrazione e quindi sarà necessario un bit per memorizzare il segno

Schema della divisione



Schema della divisione

- Il divisore V, espresso su **n** bit, viene caricato in un registro M che rimane costante per tutti i passi del calcolo.
- Il dividendo D, espresso su **2n-1** bit, viene memorizzato in A[n-2:0].Q nella fase di inizializzazione. I registri A e Q hanno parallelismo n.

  Il primo bit di A viene posto a 0 e la prima operazione è un left shift di A: in questo modo la cifra Q[0] è "libera" e può essere sostituita con la prima cifra calcolata del quoziente. Ad ogni passo i Q[0] conterrà la cifra del quoziente appena calcolata

- Se si usa la tecnica del ***restoring***, ad ogni passo dell'algoritmo viene effettuata la <u>sottrazione</u> del divisore dal dividendo parziale (shiftato) contenuto in A. Il segno della differenza determina la cifra q_i del quoziente e triggera l'eventuale operazione di restoring.

- Se si usa il metodo del ***non restoring*****,** ad ogni passo dell'algoritmo viene effettuata la <u>somma</u> o la <u>sottrazione</u> del divisore dal dividendo parziale (shiftato) contenuto in A, a seconda che il segno della precedente operazione di somma algebrica sia negativo o positivo.

- Il **segno** viene memorizzato in un flip-flop S posto in testa al registro A e viene usato per determinare la cifra q_i del quoziente.

Algoritmo di non restoring division

```
NRDivider:          (in:INBUS; OUT:OUTBUS)
                    register S,A[n-1:0],M[n-1:0],Q[n-1:0],COUNT[log2n:0];
                    bus INBUS[n-1:0], OUTBUS[n-1:0];
BEGIN:              COUNT:=0;S:=0;
INPUT:              A:=INBUS {carico la prima metà del dividendo D (0 in testa)}
                    Q=INBUS {carico la seconda metà del dividendo D}
                    M:=INBUS; {divisore V}

LSHIFT:             S.A.Q[n-1:1]=A.Q; {la prima volta S è 0, e dopo lo shift è ancora 0}

SUB:                if S==0 then
                            S.A:=S.A-M;
                    else
SUM:                        S.A:=S.A+M;
                    endif

SETq:               Q[0]:=not S;
                    COUNT:=COUNT+1;

COUNT_TEST:         if COUNT< n then goto LSHIFT;
                    endif

CORRECTION :        if S==1 then
                            S.A:=S.A+M;
                    endif

OUTPUT:     OUTBUS;=Q, OUTBUS:=A;
END NRDivider;
```